Chapter 20 — Cheat Sheets, Interview Q&A and Glossary

This last chapter is the book on a few pages. It adds no new topics. It collects the syntax, rules, decision tables and vocabulary of Chapters 1–19 in one place so you can revise quickly before an interview, or look something up while you work. Every entry links back to the section that explains it properly, with examples and real output.

What you'll find here

- **Syntax cheat sheets** for each area of the book (20.2–20.14), most ending with the traps that matter most.
- **The logical query-processing order** as a diagram (20.1). Most "why does this error happen" questions are answered by it.
- **Decision tables**: which join (20.5), which subquery form (20.6), which window function (20.8), which index (20.11), which isolation level (20.12), which scaling tool (20.14) and which database (20.15).
- **A dialect table** for PostgreSQL, MySQL, SQL Server, SQLite and Oracle (20.16).
- **Verified behaviours** (20.17). About a hundred claims from these sheets, such as `7/2 = 3`, `NULL` logic, rank ties and frame defaults, are asserted by a runnable script on PostgreSQL 14. A second script checks the SQLite column of the dialect table.
- **The interview pattern catalogue** (20.18), linking every pattern to the solved problems in Chapters 18–19.
- **Fifty interview questions with short answers** (20.19), each linked to its section.
- **A glossary** of more than 200 terms, from *1NF* to *Write skew* (20.20).

How to use it: read a sheet, cover the right-hand column of a table and say the answer out loud. Follow the link whenever an answer does not come easily. All SQL is PostgreSQL unless a row says otherwise. Other databases' syntax in the dialect table (20.16) was taken from their documentation and not run, except the SQLite column.

20.1 The logical query-processing order

You **write** `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`, but the database **evaluates** it (logically) starting from `FROM`. Each step receives the table produced by the step before it. The optimizer may physically reorder the work, but the result is always as if these steps ran in this order.

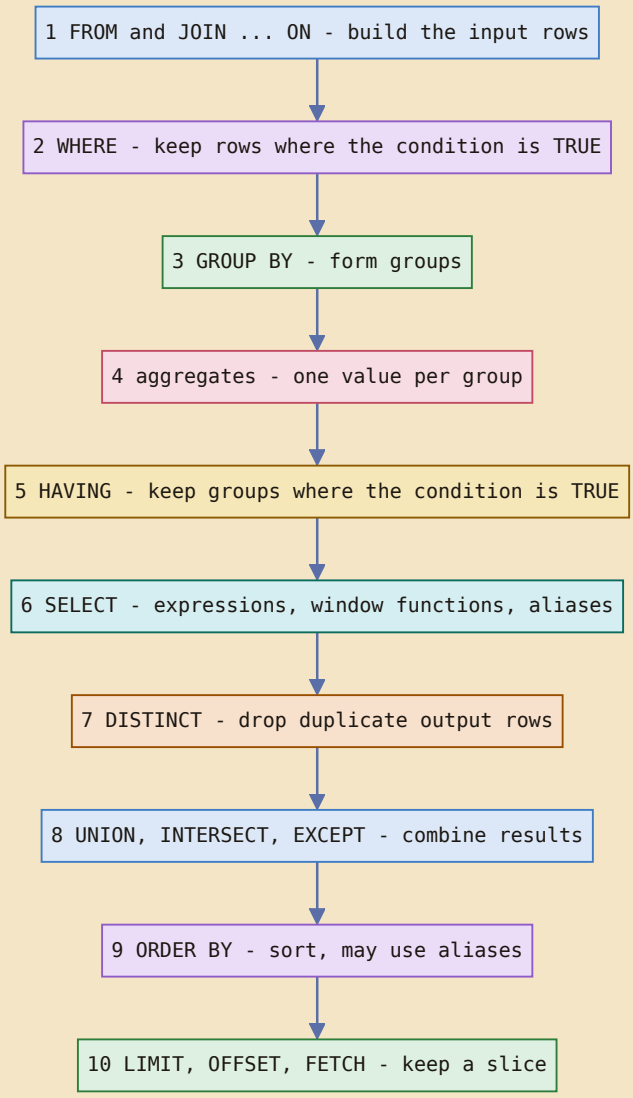


Figure 20.1 — Logical query-processing order (full explanation in §4.5 and §2.9).

Consequence	Because
`WHERE` cannot use a `SELECT` alias	aliases are created at step 6, `WHERE` is step 2
`WHERE` cannot contain `count(*)`, use `HAVING`	groups do not exist before step 3
A window function cannot appear in `WHERE` or `HAVING`	windows are computed at step 6. Filter them in an outer query or CTE (§8.10)
`ORDER BY` can use aliases and column positions	it runs after `SELECT`
`LIMIT` without `ORDER BY` returns arbitrary rows	only step 9 gives order
A filter in `WHERE` on the right table of a `LEFT JOIN` removes the padded rows	`WHERE` runs after the join has built them (§5.9)
With `SELECT DISTINCT`, `ORDER BY` may only use output columns	sorting happens after de-duplication

Two PostgreSQL relaxations: `GROUP BY` may refer to output aliases and positions (`GROUP BY 1`), and if you group by a table's primary key, the other columns of that table may be selected without being listed.

20.2 SELECT, filtering, NULL, sorting and paging

Covers Chapter 2.

```
SELECT DISTINCT e.dept_id, e.salary * 12 AS yearly     -- expressions + aliases
FROM   employees AS e                                   -- table alias
WHERE  e.salary BETWEEN 1000 AND 5000                   -- inclusive on both ends
  AND  e.dept_id IN (1, 2, 3)                           -- same as a chain of ORs
  AND  e.name ILIKE 'a%'                                -- LIKE is case-sensitive, ILIKE is not
  AND  e.manager_id IS NOT NULL                         -- never "= NULL"
ORDER  BY yearly DESC NULLS LAST, e.dept_id             -- add a tie-breaker
LIMIT  10 OFFSET 20;                                    -- or OFFSET 20 ROWS FETCH FIRST 10 ROWS ONLY
```

Need	Syntax	Remember
Rename a column	`expr AS name`	double-quote a name with spaces or capitals: `AS "Total"`
Pattern match	`LIKE 'a_%'`	`%` = any run of characters, `_` = exactly one. `ESCAPE` for literal `%`. `~` / `~*` for regular expressions
Range	`x BETWEEN a AND b`	inclusive. For timestamps use half-open `x >= a AND x < b`
NULL test	`IS NULL`, `IS NOT NULL`	`x = NULL` is never TRUE
NULL-safe compare	`a IS [NOT] DISTINCT FROM b`	treats two NULLs as equal
Default for NULL	`COALESCE(x, 0)`	returns the first non-NULL argument
Unique rows	`SELECT DISTINCT a, b`	applies to the whole output row, and NULLs count as equal
First row per group	`SELECT DISTINCT ON (g) … ORDER BY g, ts`	PostgreSQL only. The leading `ORDER BY` keys must match `g`
Sort	`ORDER BY a DESC NULLS LAST, b`	PostgreSQL puts NULLs **last** in ascending order and **first** in descending order
Page	`LIMIT n OFFSET k` / `OFFSET k ROWS FETCH FIRST n ROWS ONLY`	always with `ORDER BY`. Deep pages → keyset pagination (§12.14)
Keep ties at the cut-off	`FETCH FIRST n ROWS WITH TIES`	needs `ORDER BY` (PostgreSQL 13+)

Three-valued logic. A comparison with `NULL` gives `UNKNOWN` (shown as NULL). `WHERE` and `HAVING` keep only rows where the condition is TRUE. `CHECK` constraints reject only FALSE.

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

`NOT UNKNOWN` is UNKNOWN. Operator precedence is `NOT` > `AND` > `OR`, so add parentheses whenever you mix `AND` and `OR`.

Traps: `x <> 5` silently drops rows where `x` is NULL. `NOT IN (list containing NULL)` returns no rows (§2.5, §6.2). `BETWEEN '2024-01-01' AND '2024-01-31'` on a timestamp misses everything after midnight on the 31st. `OFFSET 100000` still reads and throws away 100,000 rows.

20.3 Functions and expressions

Covers Chapter 3.

Area	PostgreSQL	Notes
Cast	`CAST(x AS numeric(10,2))`, `x::int`	`::` is PostgreSQL-only. Bad input raises an error instead of guessing
Division	`7 / 2` → `3`, `7 / 2.0` → `3.5`, `7::numeric / 2` → `3.5`	integer ÷ integer truncates toward zero (`-7 / 2` → `-3`)
Remainder	`7 % 3`, `mod(7, 3)`	takes the sign of the dividend (`-7 % 2` → `-1`)
Rounding	`round(x, 2)`, `trunc(x)`, `ceil`, `floor`	`round(2.5)` → 3 on `numeric` (half away from zero), `round(2.5::float8)` → 2 (half to even)
Logs and powers	`ln(x)`, `log(x)` (base 10), `log(b, x)`, `power(a, b)`, `sqrt`	MySQL's `LOG(x)` is the natural log
Safe divide	`a / NULLIF(b, 0)`	`1 / 0` is an error (SQLSTATE 22012). The `NULLIF` version returns NULL. MySQL and SQLite return NULL for `x / 0` instead of failing (20.16)
Concatenate	`a || b`, `concat(a, b)`, `concat_ws(', ', a, b)`	`||` with a NULL gives NULL. `concat` skips NULLs
Length	`length(s)` / `char_length(s)` = characters, `octet_length(s)` = bytes	`'é'` is 1 character, 2 bytes in UTF-8
Pieces	`substring(s FROM 2 FOR 3)`, `left`, `right`, `split_part(s, ',', 2)`	positions are 1-based
Clean up	`trim`, `lower`, `upper`, `initcap`, `replace`, `translate`, `lpad`, `regexp_replace`	
Search	`position('x' IN s)`, `strpos(s, 'x')`	0 means "not found"
Now	`now()` = `current_timestamp` (fixed for the transaction), `clock_timestamp()` (real time)	`current_date` for today
Date arithmetic	`d + 7`, `ts + interval '2 hours'`, `date '2024-03-01' - date '2024-02-01'` → `29`	`date - date` is an integer number of days. Month addition clamps (Jan 31 + 1 month = Feb 29 in 2024)
Parts	`extract(year FROM ts)`, `date_part('dow', d)` (Sunday = 0), `date_trunc('month', ts)`, `age(a, b)`	
Format and parse	`to_char(ts, 'YYYY-MM')`, `to_date('03/05/2024', 'DD/MM/YYYY')`	
Time zones	`timestamptz` stores an instant. `ts AT TIME ZONE 'Europe/Paris'` converts	store `timestamptz`, not `timestamp`
Conditionals	`CASE WHEN … THEN … ELSE … END`, `COALESCE`, `NULLIF`, `GREATEST`, `LEAST`	no `ELSE` → NULL. PostgreSQL's `GREATEST`/`LEAST` ignore NULLs

Traps: integer averages and percentages (`100 * a / b` with integers truncates, so write `100.0 * a / b`), `float` for money (use `numeric`), `timestamp` without a time zone for events in several regions, and assuming `LOG` means the same thing in every database.

20.4 Aggregation

Covers Chapter 4.

```sql
SELECT   dept_id,
         count(*)                                AS staff,        -- rows
         count(manager_id)                       AS with_manager, -- non-NULL values
         count(DISTINCT job)                     AS jobs,
         round(avg(salary), 2)                   AS avg_salary,   -- NULLs ignored
         count(*) FILTER (WHERE salary > 5000)   AS high_paid,    -- conditional aggregate
         sum(CASE WHEN remote THEN 1 ELSE 0 END) AS remote_staff, -- portable version
         string_agg(name, ', ' ORDER BY name)    AS names
FROM     employees
WHERE    hired >= date '2020-01-01'              -- filters rows before grouping
GROUP BY dept_id
HAVING   count(*) >= 3                           -- filters groups after aggregation
ORDER BY staff DESC;
```

Function	Returns	NULL behaviour / empty input
`count(*)`	number of rows	0 for no rows
`count(x)`, `count(DISTINCT x)`	non-NULL values (distinct)	0 for no rows
`sum`, `avg`, `min`, `max`	as named	skip NULLs. **NULL** for no rows (use `COALESCE(sum(x), 0)`)
`avg(int_col)`	`numeric` in PostgreSQL	SQL Server returns an integer
`string_agg(x, sep ORDER BY …)`, `array_agg`, `json_agg`	concatenated value	`string_agg` skips NULLs
`bool_and`, `bool_or`	boolean	
`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	median (interpolated)	`percentile_disc` returns an actual value; `mode()` the most common
`GROUPING SETS ((a), (b), ())`, `ROLLUP (a, b)`, `CUBE (a, b)`	subtotals in one query	`grouping(a)` = 1 marks a rolled-up row. `ROLLUP (a, b)` = (a,b), (a), ()

Rules: every selected column must be grouped or aggregated (or be functionally dependent on a grouped primary key). NULLs form **one** group. `GROUP BY` does not sort. An aggregate query without `GROUP BY` always returns exactly one row, even over zero rows. That is why "return NULL if there is no answer" questions use an aggregate or scalar subquery (P39).

Traps: `WHERE count(*) > 1` (use `HAVING`). `count(*)` after a `LEFT JOIN` counts the padding row, so use `count(right_table.id)` to get zero (§5.9). Averages of averages. Joining two child tables before aggregating multiplies the rows (fan-out).

20.5 Joins

Covers Chapter 5.

```sql
SELECT c.name, o.id, o.total
FROM   customers c
LEFT   JOIN orders o
       ON  o.customer_id = c.id
       AND o.placed_at >= date '2024-01-01';   -- right-table filter belongs in ON
```

Join	Returns	Typical question
`CROSS JOIN`	every pair, m × n rows	"every student × every subject" grid
`[INNER] JOIN … ON`	only pairs that match	"orders with their customer"
`LEFT [OUTER] JOIN`	every left row, with matches or NULL padding	"all customers, with their orders if any"
`RIGHT [OUTER] JOIN`	mirror of `LEFT`	rewrite as `LEFT` for readability
`FULL [OUTER] JOIN`	all rows of both sides	reconciling two lists
Self join	a table joined to itself under two aliases	employee → manager, pairs, previous row
Semi-join: `EXISTS`, `IN`	left rows that have at least one match, each once	"customers who ordered"
Anti-join: `NOT EXISTS`, `LEFT JOIN … WHERE r.id IS NULL`, `EXCEPT`	left rows with no match	"customers who never ordered"
`JOIN … USING (col)`	equality on same-named columns, one output column	fine. `NATURAL JOIN` is not (it silently joins on every shared name)
`JOIN LATERAL (subquery) ON true`	per-row subquery that may reference the left row	"top 3 orders per customer" (§6.7)

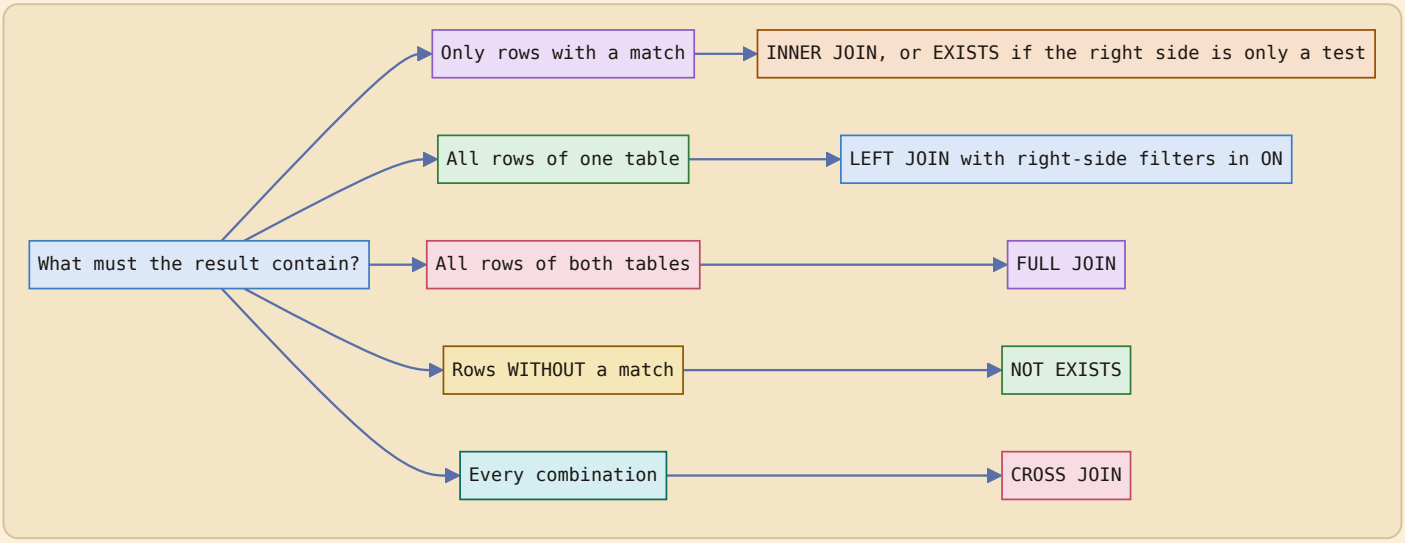


Figure 20.2 — Choosing a join.

Pitfalls (§5.9): fan-out when joining two one-to-many children, right-table conditions in `WHERE` after a `LEFT JOIN`, NULL join keys (never match), `count(*)` of padded rows, joining on names instead of keys, and `NOT IN` with NULLs. **How joins run:** nested loop (few outer rows, indexed inner key), hash join (large, equality only), merge join (both inputs sorted) (§12.15).

20.6 Subqueries and set operations

Covers Chapter 6.

Form	Example	Watch out for
Scalar subquery	`SELECT name, (SELECT max(total) FROM orders o WHERE o.cid = c.id) FROM customers c`	0 rows → NULL. 2+ rows → error
`IN` / `NOT IN`	`WHERE id IN (SELECT cid FROM orders)`	`NOT IN` returns nothing if the subquery yields a NULL
Correlated	inner query uses a column of the outer row	conceptually runs once per outer row. Qualify every column
`EXISTS` / `NOT EXISTS`	`WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cid = c.id)`	the safe anti-join. The select list inside does not matter
`ANY` / `ALL`	`salary > ALL (SELECT …)`	`> ALL (empty)` is TRUE. `> ANY (empty)` is FALSE. `= ANY` = `IN`, `<> ALL` = `NOT IN`
Derived table	`FROM (SELECT …) AS t`	alias required before PostgreSQL 16
Row subquery	`WHERE (a, b) = (SELECT x, y …)`	also `(a, b) IN (…)`
`LATERAL`	`CROSS JOIN LATERAL (SELECT … WHERE o.cid = c.id LIMIT 3) t`	`LEFT JOIN LATERAL … ON true` keeps rows with no match

Set operator	Result	Notes
`UNION`	rows in either query, duplicates removed	costs a sort or hash
`UNION ALL`	everything, duplicates kept	the default choice when duplicates are impossible or wanted
`INTERSECT [ALL]`	rows in both	NULLs compare as equal here
`EXCEPT [ALL]`	rows in the first but not the second	not commutative. Oracle calls it `MINUS`

Rules: same number of columns, compatible types, column names come from the first query, `INTERSECT` binds tighter than `UNION`/`EXCEPT`, and `ORDER BY`/`LIMIT` apply to the whole combined result (§6.11).

20.7 CTEs, recursion and views

Covers Chapter 7.

```sql
WITH RECURSIVE chain AS (
    SELECT id, manager_id, name, 1 AS depth, ARRAY[id] AS path   -- anchor: the start rows
    FROM   employees WHERE manager_id IS NULL
  UNION ALL
    SELECT e.id, e.manager_id, e.name, c.depth + 1, c.path || e.id   -- recursive term
    FROM   employees e JOIN chain c ON e.manager_id = c.id
    WHERE  NOT e.id = ANY (c.path)                                    -- cycle guard
)
SELECT repeat('  ', depth - 1) || name AS org_chart FROM chain ORDER BY path;
```

Tool	What it is	Use it for	Remember
CTE `WITH x AS (…)`	a named subquery for one statement	readable step-by-step queries, reuse by name	PostgreSQL 12+ inlines single-use CTEs. `AS MATERIALIZED` / `NOT MATERIALIZED` override
Data-modifying CTE	`WITH moved AS (DELETE … RETURNING *) INSERT … SELECT * FROM moved`	several changes in one atomic statement	PostgreSQL only. Every part sees the same snapshot
Recursive CTE	anchor `UNION [ALL]` recursive term	trees, graphs, number and date series	the recursive term sees only the previous round. Stops when a round adds no rows. Needs a stop condition
`CYCLE` / `SEARCH` clauses	built-in cycle detection and ordering	graphs	PostgreSQL 14+
`generate_series(a, b, step)`	a set-returning function	calendars, filling gaps with zeros	inclusive of both ends. With `date` arguments and an `interval` step it returns `timestamptz`, so cast with `::date`
View	a stored query, no data	simplicity, security, stable interfaces	always current. Simple one-table views are updatable. `WITH CHECK OPTION`
Materialized view	a stored result	expensive reports	stale until `REFRESH MATERIALIZED VIEW [CONCURRENTLY]` (needs a unique index)

20.8 Window functions and frames

Covers Chapter 8.

```
function(args) OVER (
    PARTITION BY dept_id                  -- independent groups; rows are NOT collapsed
    ORDER BY     hired, id                -- order inside each partition
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW   -- frame: which rows the function sees
)
```

Function	Result on scores 30, 20, 20, 10 (descending)	Uses the frame?
`row_number()`	1, 2, 3, 4 (ties broken arbitrarily unless you add a tie-breaker)	no
`rank()`	1, 2, 2, 4 (gaps after ties)	no
`dense_rank()`	1, 2, 2, 3 (no gaps)	no
`ntile(n)`	bucket numbers 1..n. Larger buckets first (7 rows into 3 → 3, 2, 2)	no
`percent_rank()` / `cume_dist()`	(rank − 1)/(n − 1) / (rows sorted before the current row + its peers)/n	no
`lag(x, k, default)` / `lead(…)`	value k rows before / after (NULL or `default` past the edge)	no
`first_value`, `last_value`, `nth_value`	value at a position **in the frame**	**yes**
`sum`, `avg`, `count`, `min`, `max` … `OVER`	aggregate over the frame	**yes**

Frame	Meaning
no `ORDER BY` in `OVER`	frame = whole partition (`sum(x) OVER ()` = grand total on every row)
`ORDER BY` without a frame clause	**`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`**, which includes all peers (ties) of the current row
`ROWS BETWEEN n PRECEDING AND CURRENT ROW`	physical rows. Use for moving averages and strict running totals
`RANGE BETWEEN interval '7 days' PRECEDING AND CURRENT ROW`	by value distance on the single `ORDER BY` key. Ties included
`GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW`	by peer groups (PostgreSQL 11+)
`EXCLUDE CURRENT ROW / GROUP / TIES`	remove rows from the frame (PostgreSQL 11+)

Patterns: running total `sum(x) OVER (PARTITION BY g ORDER BY t)`, share of total `x / sum(x) OVER ()`, change vs previous `x - lag(x) OVER (…)`, 7-row moving average with `ROWS 6 PRECEDING`, top-N per group (rank in a CTE, filter outside), de-duplication (`row_number() = 1`), gaps and islands (`value − row_number()` is constant inside a run) (§8.7–§8.9).

Traps: `last_value` with the default frame returns the current row (give it `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`). The default `RANGE` frame gives tied rows the same running total. Windows cannot be used in `WHERE`, and PostgreSQL has no `QUALIFY`. `IGNORE NULLS` arrives only in PostgreSQL 19 (in beta as of September 2026), so on 14–18 emulate it (§8.6). Each distinct window specification may need its own sort (§8.10).

20.9 DDL, DML, constraints and upsert

Covers Chapter 9.

```sql
CREATE TABLE orders (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id  bigint NOT NULL REFERENCES customers (id) ON DELETE RESTRICT,
    status       text   NOT NULL DEFAULT 'new'
                 CONSTRAINT orders_status_ck CHECK (status IN ('new', 'paid', 'shipped')),
    total        numeric(12, 2) NOT NULL CHECK (total >= 0),
    placed_at    timestamptz NOT NULL DEFAULT now(),
    UNIQUE (customer_id, placed_at)
);
CREATE INDEX ON orders (customer_id);                -- FKs are not indexed automatically

INSERT INTO orders (customer_id, total) VALUES (1, 20), (2, 35) RETURNING id;
UPDATE orders SET status = 'paid' WHERE id = 1 RETURNING *;
DELETE FROM orders o USING customers c WHERE o.customer_id = c.id AND c.banned;

INSERT INTO stock (sku, qty) VALUES ('A-1', 5)
ON CONFLICT (sku) DO UPDATE SET qty = stock.qty + EXCLUDED.qty;   -- upsert
```

Statement	Does	Remember
`CREATE TABLE [IF NOT EXISTS]`	defines a table	`CREATE TABLE t AS SELECT …` copies data but no constraints. `LIKE src INCLUDING ALL` copies structure
`ALTER TABLE … ADD / DROP / RENAME COLUMN`, `ALTER COLUMN … TYPE t USING expr`, `SET NOT NULL`, `ADD CONSTRAINT … NOT VALID` + `VALIDATE CONSTRAINT`	changes a table	many forms take an exclusive lock. Backfill before `SET NOT NULL`
`DROP TABLE [IF EXISTS] … [CASCADE]`	removes the table itself	`CASCADE` also drops dependent views and FKs
`TRUNCATE t [RESTART IDENTITY] [CASCADE]`	empties the table fast	no per-row triggers. Transactional in PostgreSQL
`DELETE FROM t WHERE …`	removes chosen rows	fires triggers, logs every row. Forget `WHERE` and every row goes
`INSERT … SELECT`	copies rows	list the target columns explicitly
`UPDATE … SET a = b, b = a`	swaps values	every `SET` expression sees the **old** row
`… RETURNING`	returns affected rows	PostgreSQL, SQLite 3.35+. SQL Server uses `OUTPUT`
`INSERT … ON CONFLICT (key) DO NOTHING / DO UPDATE SET c = EXCLUDED.c`	upsert	needs a unique index or constraint on the conflict target
`MERGE INTO t USING s ON … WHEN MATCHED … WHEN NOT MATCHED …`	standard upsert and sync	PostgreSQL 15+ (not on the book's server, 14)

Constraint	Rejects	Notes
`NOT NULL`	NULL	the most useful constraint
`PRIMARY KEY`	duplicates and NULLs	one per table. Creates a unique index
`UNIQUE`	duplicate non-NULL values	several NULLs allowed (SQL Server: one). `NULLS NOT DISTINCT` in PostgreSQL 15+
`CHECK (expr)`	rows where `expr` is FALSE	NULL passes. Cannot look at other rows
`FOREIGN KEY … REFERENCES`	child values with no parent	`ON DELETE CASCADE / SET NULL / SET DEFAULT / RESTRICT / NO ACTION` (default). NO ACTION can be deferred, RESTRICT cannot
`DEFAULT`	—	used when the column is omitted
`EXCLUDE USING gist (…)`	overlapping values	"no two bookings of one room overlap" (§17.2)

Key generation	Use
`GENERATED ALWAYS AS IDENTITY`	the standard choice. `serial` is the legacy form
Sequence (`nextval`)	shared or custom counters. Gaps are normal (rollbacks, caching)
`uuid` (`gen_random_uuid()`, PostgreSQL 13+)	ids made on many machines. Random UUIDs scatter B-tree inserts
`GENERATED ALWAYS AS (expr) STORED`	derived columns kept in sync automatically. PostgreSQL 18 adds `VIRTUAL` (computed on read), which becomes the default when neither keyword is given

Backups (§9.12): `pg_dump` makes a logical copy (plain SQL, or `-Fc` custom / `-Fd` directory formats for `pg_restore`, which can restore selected objects and run in parallel). Physical base backups + archived WAL allow point-in-time recovery (PITR). Plan by RPO (how much data you may lose) and RTO (how long recovery may take), keep 3 copies on 2 media with 1 off-site, and test restores.

20.10 Data modeling and normalization quick rules

Covers Chapter 10, Chapter 11 and Chapter 17.

Why databases exist (§1.1). File-based systems suffered from redundancy and inconsistency (the same fact stored in several files, drifting apart), isolated data in different formats, no general way to ask a new question, integrity rules buried in program code, lost updates when two users wrote at once, no atomic recovery after a crash, and all-or-nothing security. A DBMS answers each of these by mediating every access.

The three levels of a model (§10.2).

Level	Audience	Contains	Does not contain
Conceptual	business people	entities and relationships	data types, keys, DBMS details
Logical	designers	attributes, keys, cardinalities, normalized tables	a particular product's types or indexes
Physical	the DBMS	exact types, indexes, partitions, storage	—

ER notation (§10.5).

Concept	Chen notation	Crow's foot
Entity / weak entity	rectangle / double rectangle	box
Attribute	ellipse (underlined = key, double = multivalued, dashed = derived)	listed inside the box
Relationship / identifying relationship	diamond / double diamond	a line / a solid line to a weak entity
Cardinality	`1`, `N`, `M` on the lines	line ends: `||` exactly one, `o|` zero or one, `|{` one or many, `o{` zero or many
Participation	double line = total (mandatory)	inner mark: `|` mandatory, `o` optional

Mapping ER → tables (§10.15). Entity → table. Composite attribute → its parts. Multivalued attribute → its own table. Derived attribute → a view or generated column. 1:N → FK on the "many" side. M:N → junction table with a composite PK. 1:1 → FK + `UNIQUE`. Weak entity → owner's key + partial key as a composite PK, with `ON DELETE CASCADE`. Total participation → `NOT NULL` on the FK.

Keys (§10.13). Superkey (any column set that identifies a row) ⊇ candidate key (a minimal superkey) ⊇ the one chosen **primary key**. The other candidates are **alternate keys** (enforce with `UNIQUE`). A **natural key** comes from the data (email, ISBN), a **surrogate key** is made up (identity, UUID). A good default: surrogate PK + `UNIQUE` on the natural key. **Prime attribute** = belongs to some candidate key.

Functional dependencies (§11.1–§11.5). `X → Y` means rows that agree on X agree on Y. It is a business rule: data can disprove it but never prove it. **Closure X^+**: start with X, fire every FD whose left side is fully inside the set, repeat until nothing changes. X is a superkey when X^+ contains every attribute. **Candidate keys**: attributes that never appear on a right side must be in every key, so start from them. **Armstrong's axioms**: reflexivity, augmentation, transitivity (plus derived union, decomposition, pseudotransitivity). **Minimal cover**: single-attribute right sides, no extraneous left-side attributes, no redundant FDs.

Normal forms (§11.6–§11.13).

Form	Rule (informal)	Removes
1NF	every cell holds one atomic value, no repeating groups	lists in a column, `phone1, phone2, phone3`
2NF	1NF + no non-prime attribute depends on **part** of a composite key	partial dependencies
3NF	2NF + no non-prime attribute depends on another non-key attribute. Formally: for each FD X → A, X is a superkey or A is prime	transitive dependencies
BCNF	for every non-trivial FD X → Y, X is a superkey	the remaining FD redundancy
4NF	BCNF + no non-trivial multivalued dependency unless its left side is a superkey	independent lists stored together
5NF	every join dependency follows from the keys	facts that only make sense as three-way combinations

Two things people often get wrong about 1NF: atomicity depends on how the data is **used** (a full name is atomic if you never query first names separately), and a table is not in 1NF just because each cell holds one string. A comma-separated list in one string still breaks it.

Anomalies (why normalize): *insertion* (cannot record a fact without an unrelated one), *update* (one fact stored in many rows, so a change must touch all of them), *deletion* (removing one fact loses another). **Every decomposition must be lossless** (the shared columns are a key of at least one piece) and ideally **dependency-preserving**. 3NF can always have both. BCNF sometimes loses an FD, so most designs stop at 3NF (§11.10).

Design judgement calls (§17.14).

Decision	Rule of thumb
Type vs instance	separate the catalogue row from the physical item (title vs copy, room type vs room, seat vs seat-for-a-show)
Current state vs history	a status column for "now" plus an append-only events or ledger table for "what happened"
Snapshot vs reference	copy values that must not change later (price paid, shipping address) into the order row. That is a separate fact, not redundancy
Delete vs soft-delete	soft-delete (`deleted_at`) rows that history still references
Enforce where	rules in the schema (`CHECK` , `UNIQUE` , partial unique index, `EXCLUDE`) rather than only in application code
Denormalize when	only for a measured read problem, updated in the same transaction, with a reconciliation query (§11.16)
Scarce items	a guarded `UPDATE … WHERE status = 'free'` in a short transaction, or `FOR UPDATE SKIP LOCKED`

20.11 Indexes and reading EXPLAIN

Covers Chapter 12.

```
CREATE INDEX CONCURRENTLY orders_cust_date_ix ON orders (customer_id, placed_at DESC); -- composite
CREATE INDEX ON orders (customer_id) INCLUDE (total);              -- covering (index-only scans)
CREATE INDEX ON orders (placed_at) WHERE status = 'new';           -- partial
CREATE INDEX ON users (lower(email));                              -- expression
CREATE INDEX ON docs USING gin (body_tsv);                         -- full-text / jsonb / arrays
CREATE INDEX ON readings USING brin (taken_at);                    -- huge, append-only, time-ordered
EXPLAIN (ANALYZE, BUFFERS) SELECT … ;                              -- runs the query and shows the plan
```

Which index?

Query shape	Index	Why
`=`, `<`, `>`, `BETWEEN`, `ORDER BY`, `LIKE 'abc%'` (with `text_pattern_ops` or C collation)	**B-tree** (default)	sorted, supports ranges and order
`WHERE a = ? AND b > ?`	composite B-tree `(a, b)`	equality columns first, range column last
`WHERE a = ?` returning only a few columns	`(a) INCLUDE (x, y)`	index-only scan, no table visits
Only a small hot subset is queried (`status = 'new'`)	partial index `WHERE status = 'new'`	smaller, cheaper to maintain
`WHERE lower(email) = ?`	expression index on `lower(email)`	the index must match the expression
Equality only, very large keys	hash	cannot do ranges or ordering
JSONB `@>`, arrays `&&` / `@>`, full-text `@@`, trigram `LIKE '%x%'`	**GIN** (`pg_trgm` for trigrams)	inverted index: value → rows
Ranges overlapping, geometry, nearest neighbour, `EXCLUDE` constraints	**GiST**	tree of bounding boxes
Billions of rows physically ordered by time	**BRIN**	a few bytes per block range
Low-selectivity column (`is_active` true for 90 %)	usually **none**	a sequential scan is cheaper

Reading a plan (§12.10). Read from the innermost node outward. Each node shows `cost=startup..total` (planner units), estimated `rows`, and, with `ANALYZE`, `actual time`, actual `rows` and `loops` (multiply per-loop numbers by loops). `BUFFERS` shows pages hit in memory vs read.

Node	Means	Chosen when
Seq Scan	reads every page	a large fraction of the table is needed, or no usable index
Index Scan	walks the index, then fetches each row	few rows, or rows needed in index order
Index Only Scan	answers from the index alone	all needed columns are in the index and pages are all-visible (vacuumed)
Bitmap Index Scan + Bitmap Heap Scan	collects row locations, then reads pages in order	a medium number of scattered rows. Can combine indexes with AND/OR
Nested Loop	for each outer row, probe the inner	small outer side + index on the inner join key
Hash Join	builds a hash table on the smaller input	large inputs, equality joins
Merge Join	merges two sorted inputs	both sides already sorted (or cheap to sort)
Sort / Incremental Sort, HashAggregate / GroupAggregate	sorting and grouping	watch `Sort Method: external merge` (spilled to disk)

The big warning sign: estimated rows far from actual rows. Refresh statistics with `ANALYZE`, and consider extended statistics for correlated columns (§12.16).

Anti-patterns that disable an index (§12.13): a function or arithmetic on the indexed column (`WHERE date(ts) = …`, `WHERE price * 1.2 > …`), a leading wildcard (`LIKE '%x'`), implicit casts, `OR` across different columns (sometimes fixable with `UNION` or a bitmap), a composite index whose leading column is not filtered (PostgreSQL 18+, like Oracle and MySQL 8.0.13+, can sometimes skip-scan it when that column has few distinct values, but don't design around it), deep `OFFSET` paging (use keyset: `WHERE (ts, id) < (:last_ts, :last_id) ORDER BY ts DESC, id DESC LIMIT 20`), and N+1 query loops from application code. Every index slows writes and uses space, so index real query patterns, not columns.

20.12 Transactions, isolation and concurrency

Covers Chapter 13.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
  SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;   -- lock the row you will change
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  SAVEPOINT s1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
  -- ROLLBACK TO SAVEPOINT s1;   -- undo only the part after s1
COMMIT;                          -- or ROLLBACK
```

ACID. *Atomicity*: all or nothing (undo via log or, in PostgreSQL, by marking the transaction aborted). *Consistency*: constraints and application rules hold before and after. *Isolation*: concurrent transactions do not see each other's partial work (locks, MVCC, SSI). *Durability*: a committed change survives a crash (write-ahead log flushed at commit, checkpoints, replicas).

Anomalies (§13.4).

Anomaly	What happens	Example
Dirty write	overwrite another transaction's uncommitted change	two uncommitted updates on one row
Dirty read	read another transaction's uncommitted change	see a balance that is later rolled back
Non-repeatable read (read skew)	the same row read twice gives different values. Read skew: different rows seen at different moments	a report sees an account before a transfer and the other account after it
Phantom read	the same query returns a different **set** of rows	a count rises because someone inserted a matching row
Lost update	two read-modify-write cycles, the second overwrites the first	two sales both set stock 10 → 9
Write skew	two transactions read the same rows, then each updates a **different** row, breaking a rule both checked	two doctors both go off call, leaving nobody on call

Isolation levels — standard minimum vs PostgreSQL.

Level	Standard forbids	PostgreSQL actually prevents	Still possible in PostgreSQL
Read Uncommitted	—	dirty reads (it behaves as Read Committed)	as Read Committed
Read Committed (PostgreSQL default)	dirty read	dirty read and write	non-repeatable read, phantom, lost update, write skew
Repeatable Read (snapshot isolation)	+ non-repeatable read	+ phantoms. Lost updates abort with SQLSTATE 40001	write skew
Serializable (SSI)	+ phantom	everything. One transaction aborts with 40001	nothing, but you must retry

Which isolation level (and lock)?

Situation	Choose
Ordinary OLTP, single-statement changes (`SET qty = qty - 1`)	Read Committed
A consistent multi-query report or export	Repeatable Read, `READ ONLY`
Read a row, decide, write it back	`SELECT … FOR UPDATE` at Read Committed, or an atomic `UPDATE … WHERE`, or Repeatable Read + retry
Optimistic editing across user think-time	a `version` column: `UPDATE … SET version = version + 1 WHERE id = ? AND version = ?`
A rule spanning several rows (no double booking, one on call)	Serializable + retry, an `EXCLUDE`/unique constraint, or explicit locks
Many workers taking jobs from a queue	`SELECT … FOR UPDATE SKIP LOCKED LIMIT 1`
Must not wait at all	`FOR UPDATE NOWAIT` (errors immediately if locked)

Concurrency control families (§13.9). Pessimistic locking (shared/exclusive locks, two-phase locking: grow then shrink, strict 2PL holds write locks to commit). Optimistic (work freely, validate at commit, abort on conflict). Timestamp ordering (each transaction gets a timestamp, operations out of order are rejected). **MVCC** (writers create new row versions, readers see a snapshot, so readers never block writers. PostgreSQL marks versions with `xmin`/`xmax` and cleans dead ones with VACUUM).

Deadlocks (§13.11): two transactions each wait for a lock the other holds. PostgreSQL detects it after `deadlock_timeout` and aborts one (SQLSTATE 40P01). Prevent it by locking rows in a consistent order and keeping transactions short. Application code should **retry on 40001 and 40P01**.

20.13 Advanced PostgreSQL: PL/pgSQL, JSONB, full-text search, security

Covers Chapter 14.

```
CREATE FUNCTION order_total(p_id bigint) RETURNS numeric
LANGUAGE sql STABLE AS $$ SELECT sum(qty * price) FROM order_lines WHERE order_id = p_id $$;

CREATE PROCEDURE archive_old() LANGUAGE plpgsql AS $$
BEGIN
  LOOP
    DELETE FROM events WHERE id IN (SELECT id FROM events WHERE ts < now() - interval '1 year' LIMIT
10000);
    EXIT WHEN NOT FOUND;
    COMMIT;                                  -- procedures may commit (when CALLed outside a transaction)
  END LOOP;
END $$;
CALL archive_old();

CREATE FUNCTION touch() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$;
CREATE TRIGGER t_touch BEFORE UPDATE ON products FOR EACH ROW EXECUTE FUNCTION touch();
```

Feature	Syntax	Remember
Function	`CREATE FUNCTION f(args) RETURNS t LANGUAGE sql / plpgsql`	declare volatility honestly: `IMMUTABLE`, `STABLE`, `VOLATILE` (default)
Procedure	`CREATE PROCEDURE`, `CALL p()`	can `COMMIT` / `ROLLBACK` inside. `INOUT` parameters
Trigger	`BEFORE` / `AFTER` / `INSTEAD OF`, `FOR EACH ROW` / `STATEMENT`	BEFORE can modify `NEW` or reject (return NULL). AFTER reacts. Keep them fast
JSONB	`->` (jsonb), `->>` (text), `#>> '{a,b}'`, `@>` (contains), `?` (key exists), `jsonb_set`, `jsonb_each`, `jsonb_array_elements`	binary, keys de-duplicated (last wins) and reordered. GIN index for `@>` and `?`. In a quick demo, `VACUUM` after bulk updates before building the index, or older snapshots may ignore it
Arrays	`ARRAY[1,2]`, `a[1]` (1-based), `= ANY(a)`, `@>`, `&&`, `unnest`, `array_agg`	out-of-range subscript returns NULL
Full-text search	`to_tsvector('english', body) @@ websearch_to_tsquery('english', 'fast db')`, `ts_rank`	stemming and stop words. Store a generated `tsvector` column with a GIN index
Pivot	`max(v) FILTER (WHERE k = 'a') AS a … GROUP BY id`, or `crosstab` (tablefunc)	unpivot with `CROSS JOIN LATERAL (VALUES ('a', a), ('b', b))`
Roles and privileges	`CREATE ROLE app LOGIN`, `GRANT SELECT, INSERT ON t TO app`, `REVOKE`	group roles, least privilege
Row-level security	`ALTER TABLE t ENABLE ROW LEVEL SECURITY`, `CREATE POLICY p ON t USING (tenant_id = current_setting('app.tenant')::int)`	`USING` filters reads, `WITH CHECK` filters writes. Superusers and `BYPASSRLS` roles always bypass it; the table owner does too unless `FORCE ROW LEVEL SECURITY`
SQL injection defence	bind parameters: psycopg `%s`, sqlite3 `?`, JDBC `PreparedStatement` `?`	never concatenate user input. Identifiers: allow-list or `format('%I', …)`
From code	explicit transactions, `Decimal`/`BigDecimal` for money, batching or `COPY` for bulk loads, stream large results	a connection pool (small, reused) or PgBouncer in front of many clients
ORMs	fast for CRUD	watch for N+1 queries. Hand-write complex and hot queries (§14.13)

20.14 Scaling: partitioning, sharding, replication

Covers Chapter 15.

```
CREATE TABLE events (id bigint, ts timestamptz NOT NULL, payload jsonb)
PARTITION BY RANGE (ts);
CREATE TABLE events_2024_06 PARTITION OF events
  FOR VALUES FROM ('2024-06-01') TO ('2024-07-01');       -- upper bound is exclusive
-- also PARTITION BY LIST (region) and PARTITION BY HASH (id) … FOR VALUES WITH (MODULUS 4, REMAINDER 0)
```

Tool	What it does	Choose it when	Cost
Scale up	a bigger server	first, almost always	a ceiling, one failure domain
Read replicas	copies that serve reads	read-heavy workloads, reporting	replication lag, stale reads
Partitioning (one server)	split a table by range, list or hash	huge tables, time-based retention (drop a partition), pruning	unique keys must include the partition key
Sharding (many servers)	partitions on different machines	writes or data exceed one server	cross-shard joins and transactions, rebalancing, operations
Caching	keep hot results in memory	repeated reads of the same data	invalidation
Denormalized / summary tables	precomputed answers	expensive repeated aggregates	keeping copies correct

Concept	One-line rule
Shard key	high cardinality, spreads load evenly, matches the most common query filter
Consistent hashing	only about 1/n of keys move when a node joins. Virtual nodes smooth the load
Synchronous vs asynchronous replication	sync = no committed data lost but slower writes. Async = fast, with lag and possible loss on failover
Single-leader / multi-leader / leaderless	one writer (simple) / several writers (conflicts to resolve) / quorums with R + W > N
Consistency models	strong (linearizable) > sequential > causal > session (read-your-writes) > eventual
CAP	during a network partition choose consistency or availability
PACELC	if Partitioned choose A or C, Else choose Latency or Consistency
Consensus (Raft, Paxos)	a majority of 2f + 1 nodes tolerates f crashes. Byzantine (PBFT) needs 3f + 1
Two-phase commit	prepare, then commit. Blocks if the coordinator dies after prepare
Recovery	WAL first, checkpoints bound replay, redo committed work, undo uncommitted work
High availability	heartbeat → quorum decides → promote a replica → fence the old leader → redirect clients
Backups vs replicas	a replica copies your mistakes too. Only backups (with PITR) undo a bad `DELETE`

20.15 Which database?

Covers Chapter 16. Default to PostgreSQL for transactional data and add a specialised store only when a requirement clearly exceeds it (§16.10).

Family	Data shape	Strength	Weakness	Examples
Relational (OLTP)	tables, keys, constraints	joins, ACID, ad-hoc SQL	scaling writes beyond one node takes work	PostgreSQL, MySQL, SQL Server, Oracle
Key-value	key → opaque value	fastest get/put, TTL, simple scale-out	no queries by value	Redis, DynamoDB (also document-like), Memcached
Document	nested JSON per record	flexible schema, whole-object reads	cross-document joins and transactions are limited or costly	MongoDB, Couchbase, Firestore
Wide-column	partition key + sorted clustering columns	huge write rates, time-ordered data per key	must model around queries, no joins	Cassandra, ScyllaDB, HBase, Bigtable
Graph	nodes and edges	multi-hop traversals, paths	bulk analytics over all data, sharding	Neo4j, Amazon Neptune
Time-series	timestamped measurements	compression, retention, downsampling	general-purpose queries	TimescaleDB, InfluxDB, Prometheus
Search engine	inverted index of documents	relevance ranking, facets, fuzzy text	not a system of record	Elasticsearch, OpenSearch
Columnar warehouse	column-oriented tables	scans and aggregates over billions of rows	slow single-row writes	BigQuery, Snowflake, Redshift, ClickHouse
NewSQL / distributed SQL	SQL tables on consensus-replicated shards	SQL + ACID across nodes and regions	latency of consensus, operational complexity	Spanner, CockroachDB, YugabyteDB, TiDB
In-memory	data kept in RAM, persisted by snapshot + log	microsecond latency	cost per GB, durability settings	Redis, SAP HANA, VoltDB

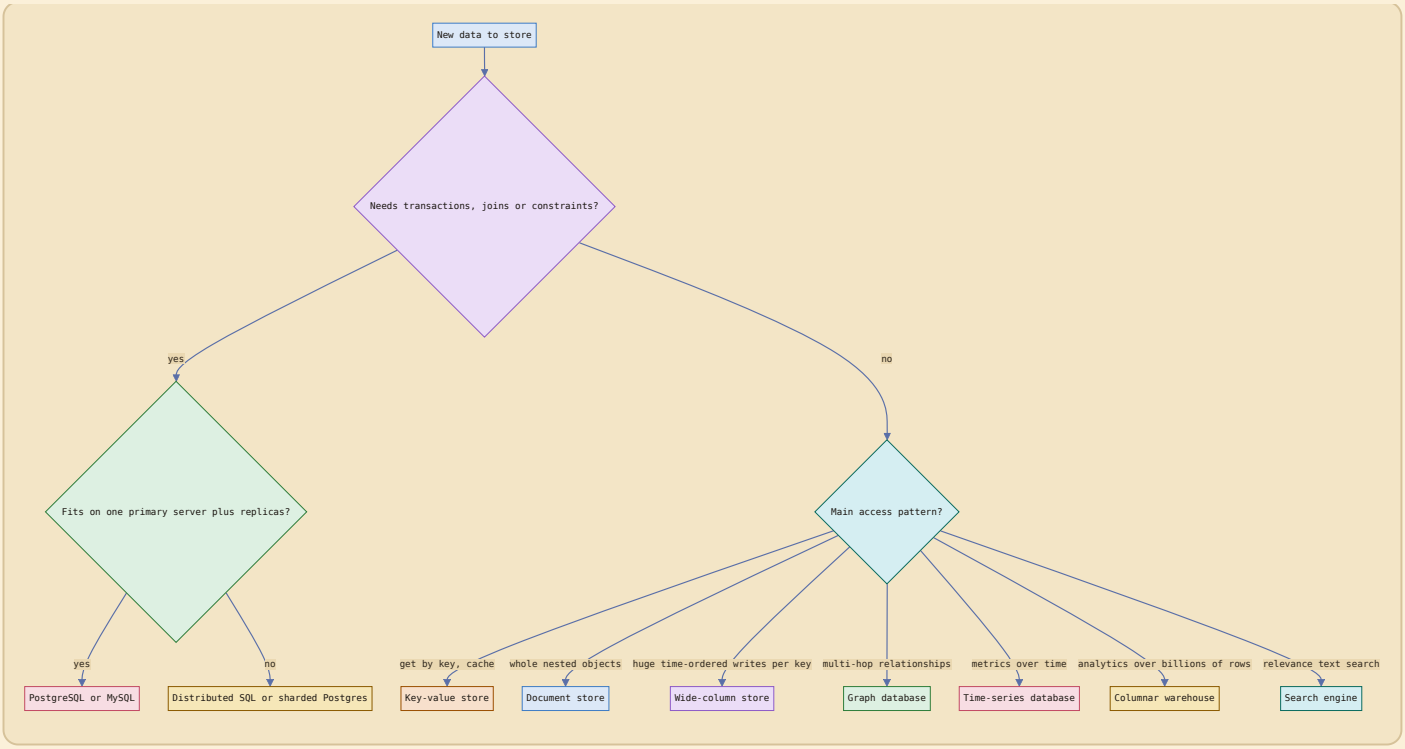


Figure 20.3 — A first-pass database choice. Real systems often combine several (polyglot persistence), fed from one source of truth by change data capture.

ACID vs BASE. BASE = *Basically Available, Soft state, Eventually consistent*: stay up and accept writes, and let replicas converge later. It is a spectrum, not a switch: many NoSQL stores offer tunable or per-operation consistency.

20.16 Dialect differences

The book runs PostgreSQL 14. This table collects the differences you are most likely to hit when you move a query to another database. Versions in brackets are the first release with the feature. "—" means there is no built-in equivalent. Oracle means 19c, with features added in 21c or 23ai (renamed 26ai) marked. The SQLite column was checked on SQLite 3.43 by `code/02_sqlite_dialect.sqlite.sql` (20.17). The other columns come from each vendor's documentation and were not run here.

Syntax and basics

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
First n rows	`LIMIT n` / `FETCH FIRST n ROWS ONLY`	`LIMIT n`	`TOP (n)` / `OFFSET 0 ROWS FETCH NEXT n ROWS ONLY` (2012, needs `ORDER BY`)	`LIMIT n`	`FETCH FIRST n ROWS ONLY` (12c). Older: `ROWNUM`
Quote an identifier	`"Name"`	backticks around the name (`"` only in ANSI_QUOTES mode)	`[Name]` or `"Name"`	`"Name"` (backticks and `[Name]` also accepted)	`"Name"`
Unquoted names are folded to	lower case	case of table names depends on the OS. Column names are case-insensitive	kept. Case-sensitivity follows the collation (insensitive by default)	kept, compared case-insensitively	UPPER case
String concatenation	`a || b` (NULL if either is NULL), `concat()`	`CONCAT(a, b)` (NULL if any is NULL). `||` means OR by default	`a + b`, `CONCAT()` (NULL treated as '')	`a || b`	`a || b` (NULL treated as '')
Empty string `''`	a value, not NULL	a value	a value	a value	**is NULL**
`7 / 2`	`3`	`3.5000` (`7 DIV 2` = 3)	`3`	`3`	`3.5`
`1 / 0` in a `SELECT`	error (22012)	NULL + warning (strict mode errors only in `INSERT`/`UPDATE`)	error (8134)	NULL	error (ORA-01476)
Cast	`CAST(x AS t)`, `x::t`	`CAST`, `CONVERT(x, t)`	`CAST`, `CONVERT(t, x)`, `TRY_CAST`	`CAST`	`CAST`, `TO_NUMBER`, `TO_DATE`
Boolean type	`boolean`	`BOOLEAN` = `TINYINT(1)`	`BIT`	integers 0/1 (keywords `TRUE`/`FALSE` accepted)	`BOOLEAN` (23ai). Before: none in SQL
`SELECT` without `FROM`	yes	yes (or `FROM DUAL`)	yes	yes	`FROM DUAL` required before 23ai
Type checking	strict	lenient unless strict SQL mode (the default in 8.0)	strict	dynamic typing (type affinity). `STRICT` tables (3.37)	strict
Auto-increment	`GENERATED … AS IDENTITY`, `serial`	`AUTO_INCREMENT`	`IDENTITY(1,1)`	`INTEGER PRIMARY KEY` (optionally `AUTOINCREMENT`)	`GENERATED … AS IDENTITY` (12c), sequences
Sequences	yes	—	yes (2012)	—	yes
`DROP TABLE IF EXISTS`	yes	yes	yes (2016)	yes	23ai

NULLs, comparison and sorting

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
NULL-safe equality	`IS NOT DISTINCT FROM`	`<=>`	`IS NOT DISTINCT FROM` (2022)	`IS` (and `IS NOT DISTINCT FROM`, 3.39)	— (use `DECODE(a, b, 1, 0) = 1`)
NULLs in `ORDER BY … ASC`	last	first	first	first	last
`NULLS FIRST / LAST`	yes	— (sort by `x IS NULL` first)	— (sort by a `CASE` first)	yes (3.30)	yes
First non-NULL	`COALESCE`	`COALESCE`, `IFNULL`	`COALESCE`, `ISNULL`	`COALESCE`, `IFNULL`	`COALESCE`, `NVL`
`GREATEST(1, NULL)`	1 (NULLs ignored)	NULL	1 (2022)	use `max(a, b)` (NULL if any NULL)	NULL
`UNIQUE` column with several NULLs	allowed (`NULLS NOT DISTINCT` 15+)	allowed	**one** NULL only (use a filtered index)	allowed	allowed (all-NULL keys are not indexed)
`LIKE` case-sensitive?	yes (`ILIKE` for no)	no with the default collation	no with the default collation	no for ASCII	yes
Regular expressions	`~`, `~*`, `regexp_replace`	`REGEXP`, `REGEXP_LIKE` (8.0)	`REGEXP_LIKE`, `REGEXP_REPLACE` (2025). Before: `LIKE` patterns only	`REGEXP` only if a function is loaded (the shell has one)	`REGEXP_LIKE`, `REGEXP_REPLACE`

Joins, subqueries and set operations

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
`FULL OUTER JOIN`	yes	— (emulate with `UNION`)	yes	yes (3.39)	yes
`RIGHT JOIN`	yes	yes	yes	yes (3.39)	yes
`USING`, `NATURAL JOIN`	yes	yes	—	yes	yes
`LATERAL`	yes	yes (8.0.14)	`CROSS APPLY` / `OUTER APPLY`	—	`LATERAL`, `CROSS/OUTER APPLY` (12c)
`INTERSECT`, `EXCEPT`	yes	yes (8.0.31)	yes	yes	`INTERSECT`, `MINUS` (`EXCEPT` 21c)
Row values `(a, b) IN (SELECT …)`	yes	yes	— (use `EXISTS`)	yes (3.15)	yes
`UPDATE` with a join	`UPDATE t SET … FROM s WHERE …`	`UPDATE t JOIN s ON … SET …`	`UPDATE t SET … FROM t JOIN s ON …`	`UPDATE … FROM` (3.33)	`MERGE`, or a correlated subquery (direct joins 23ai)

Aggregation, CTEs and windows

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
String aggregation	`string_agg(x, ',' ORDER BY x)`	`GROUP_CONCAT(x ORDER BY x SEPARATOR ',')`	`STRING_AGG(x, ',') WITHIN GROUP (ORDER BY x)` (2017)	`group_concat(x, ',')` (`string_agg` 3.44)	`LISTAGG(x, ',') WITHIN GROUP (ORDER BY x)`
`FILTER (WHERE …)` on aggregates	yes	— (use `CASE`)	— (use `CASE`)	yes (3.30)	— (use `CASE`)
`avg()` of an integer column	exact `numeric`	decimal	**integer** (truncated)	floating point	exact `NUMBER`
Median	`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	—	`PERCENTILE_CONT` as a window function only	—	`MEDIAN(x)`, `PERCENTILE_CONT`
`ROLLUP` / `CUBE` / `GROUPING SETS`	all three	`WITH ROLLUP` only	all three	—	all three
Recursive CTE keyword	`WITH RECURSIVE`	`WITH RECURSIVE`	plain `WITH`	`WITH RECURSIVE`	plain `WITH` (column list required)
Window functions	yes (8.4)	yes (8.0)	ranking 2005, frames 2012	yes (3.25)	yes (8i)
`GROUPS` frames, `EXCLUDE`	yes (11)	—	—	yes (3.28)	yes (21c)
`WINDOW w AS (…)` clause	yes	yes	yes (2022)	yes	yes (21c)
`IGNORE NULLS` in `lag` / `first_value`	— (19, in beta as of September 2026)	—	yes (2022)	—	yes
`QUALIFY`	—	—	—	—	26ai
`DISTINCT ON`	yes	—	—	—	—

DDL, DML and transactions

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Upsert	`INSERT … ON CONFLICT`, `MERGE` (15)	`INSERT … ON DUPLICATE KEY UPDATE`, `REPLACE`, `INSERT IGNORE`	`MERGE`	`INSERT … ON CONFLICT` (3.24), `INSERT OR REPLACE`	`MERGE`
Return changed rows	`RETURNING`	—	`OUTPUT inserted.*`	`RETURNING` (3.35)	`RETURNING … INTO` (PL/SQL)
DDL inside a transaction	yes, can be rolled back	no, causes an implicit commit	mostly yes	yes	no, DDL commits
`CHECK` enforced	yes	yes (8.0.16; parsed but ignored before)	yes	yes	yes
Default isolation level	Read Committed	Repeatable Read	Read Committed (locking)	Serializable	Read Committed
"Serializable" means	SSI (true serializability)	2PL: plain reads become locking reads	2PL (or `SNAPSHOT` level)	one writer at a time	snapshot isolation (write skew possible)
Stored-procedure language	PL/pgSQL (and others)	SQL/PSM-style	T-SQL	— (host language)	PL/SQL
Materialized views	yes, manual `REFRESH`	—	indexed views	—	yes, with incremental (fast) refresh

Indexes and plans

Feature	PostgreSQL	MySQL 8 (InnoDB)	SQL Server	SQLite	Oracle
Table storage	heap + separate indexes	clustered on the primary key	heap or clustered index	B-tree keyed by rowid	heap (index-organized tables optional)
Partial index	yes	—	filtered index	yes	— (function-based index returning NULL)
Covering `INCLUDE`	yes (11)	— (add to the key)	yes	—	—
Expression index	yes	functional key parts (8.0.13)	index a computed column	yes	function-based index
Show the plan	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE` (8.0.18), `EXPLAIN FORMAT=TREE`	actual execution plan, `SET STATISTICS IO ON`	`EXPLAIN QUERY PLAN`	`EXPLAIN PLAN FOR` + `DBMS_XPLAN.DISPLAY`

Dates and functions

Task	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Now	`now()`	`NOW()`	`SYSDATETIME()`, `GETDATE()`	`datetime('now')`	`SYSTIMESTAMP`, `SYSDATE`
Add 3 days	`d + 3`, `ts + interval '3 days'`	`DATE_ADD(d, INTERVAL 3 DAY)`	`DATEADD(day, 3, d)`	`date(d, '+3 days')`	`d + 3`, `d + INTERVAL '3' DAY`
Days between	`d2 - d1`	`DATEDIFF(d2, d1)`	`DATEDIFF(day, d1, d2)`	`julianday(d2) - julianday(d1)`	`d2 - d1`
Truncate to month	`date_trunc('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`DATETRUNC(month, d)` (2022)	`date(d, 'start of month')`	`TRUNC(d, 'MM')`
Year of a date	`extract(year FROM d)`	`YEAR(d)`	`YEAR(d)`	`strftime('%Y', d)`	`EXTRACT(YEAR FROM d)`
Characters / bytes	`length` / `octet_length`	`CHAR_LENGTH` / `LENGTH`	`LEN` (ignores trailing spaces) / `DATALENGTH`	`length` / `length(CAST(s AS BLOB))`	`LENGTH` / `LENGTHB`
If-then-else shortcut	`CASE`	`IF(c, a, b)`	`IIF(c, a, b)`	`iif(c, a, b)` (3.32)	`DECODE`, `CASE`
Random number 0–1	`random()`	`RAND()`	`RAND()`	`random() / 18446744073709551616.0 + 0.5`	`DBMS_RANDOM.VALUE`
JSON	`json`, `jsonb` (+ GIN)	`JSON` (binary)	`nvarchar` + `JSON_VALUE` etc. (native `json` type 2025)	JSON functions on text (built in since 3.38)	`JSON` type (21c)
Arrays	yes	–	–	–	collection types (`VARRAY`) in PL/SQL

20.17 Verified behaviours

Every concrete behaviour quoted on these sheets, such as `7 / 2 = 3`, `NULL = NULL` being unknown, rank ties, the default window frame, `NOT IN` with a NULL, upsert, and `->>` returning text, is asserted by `code/01_cheatsheet_checks.sql`. The fixtures and a small checker live in `code/00_schema.sql`:

```
CREATE FUNCTION chk(p_area text, p_claim text, p_got text, p_expected text)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  IF p_got IS DISTINCT FROM p_expected THEN
    RAISE EXCEPTION 'cheat-sheet claim FAILED [%] %: got %, expected %',
      p_area, p_claim, coalesce(p_got, 'NULL'), coalesce(p_expected, 'NULL');
  END IF;
  INSERT INTO check_log (area, claim, got, expected)
  VALUES (p_area, p_claim, coalesce(p_got, 'NULL'), coalesce(p_expected, 'NULL'));
END $$;
```

`chk` compares with `IS DISTINCT FROM`, so an expected NULL is checked properly too (a plain `<>` would let a NULL result slip through, which is exactly the trap of 20.2). The checks are grouped into `DO` blocks by area. A few examples:

```
DO $$ BEGIN
  PERFORM chk('NULL', '2 NOT IN (1, NULL) is NULL, not TRUE',     (2 NOT IN (1, NULL))::text, NULL);
  PERFORM chk('NULL', 'NOT IN over a list holding NULL keeps 0 rows',
              (SELECT count(*) FROM customers WHERE id NOT IN (SELECT customer_id FROM banned))::text,
'0');
  -- ...
END $$;

DO $$ BEGIN
  PERFORM chk('Windows', 'row_number / rank / dense_rank on 30,20,20,10',
              (SELECT string_agg(format('%s:%s/%s/%s', name, rn, rk, drk), ' ' ORDER BY rn)
                 FROM (SELECT name,
                              row_number() OVER (ORDER BY points DESC, name) AS rn,
                              rank()       OVER (ORDER BY points DESC)       AS rk,
                              dense_rank() OVER (ORDER BY points DESC)       AS drk
                         FROM scores WHERE points IS NOT NULL) t),
              'Al:1/1/1 Bo:2/2/2 Cy:3/2/2 Di:4/4/3');
  PERFORM chk('Windows', 'default frame with ORDER BY is RANGE: peers share the running total',
              (SELECT string_agg(s::text, ',' ORDER BY day, amount)
                 FROM (SELECT day, amount, sum(amount) OVER (ORDER BY day) AS s FROM sales) t),
              '10,40,40,45');
  -- ...
END $$;

-- transactional DDL: a TRUNCATE inside a failed sub-block is rolled back
BEGIN
  TRUNCATE t_users;
  RAISE EXCEPTION 'undo';
EXCEPTION WHEN raise_exception THEN NULL;
END;
PERFORM chk('DDL', 'TRUNCATE is transactional in PostgreSQL (rows came back)',
            (SELECT count(*) FROM t_users)::text, '3');
```

If a claim were wrong, the file would stop with, for example, `ERROR: cheat-sheet claim FAILED [Functions] …: got 2/2, expected 1/2`. (That exact message appeared once while this chapter was being written, when the scratch database was accidentally created with the `SQL_ASCII` encoding. In that encoding `length('é')` counts bytes. The runner always creates UTF-8 databases.) The file ends by listing what was verified.

Output (`bash tools_run_sql.sh sql '20-*'` → `OK`, and the final queries):

```
     area     | claims
--------------+--------
 NULL         |     18
 Functions    |     18
 Aggregation  |     12
 Joins        |      5
 Subqueries   |      3
 Set ops      |      5
 DISTINCT     |      1
 ORDER BY     |      4
 WHERE        |      4
 Windows      |     12
 CTEs         |      3
 Constraints  |      3
 Upsert       |      2
 DML          |      1
 DDL          |      1
 Transactions |      2
 JSONB        |      5
 Arrays       |      3
 FTS          |      2
(19 rows)

 total_claims_verified
-----------------------
                   104
(1 row)
```

The SQLite column of the dialect table is checked the same way by `code/02_sqlite_dialect.sqlite.sql`. SQLite has no `DO` blocks, so each claim is inserted into a table whose `CHECK (ok = 1)` fails the script when a claim is false:

```sql
CREATE TABLE claim (what TEXT, ok INTEGER CHECK (ok = 1));
INSERT INTO claim VALUES
 ('7/2 = 3 (integer division)',              7 / 2 = 3),
 ('a || NULL is NULL',                       ('a' || NULL) IS NULL),
 ('LIKE is case-insensitive for ASCII',      'Anna' LIKE 'a%'),
 ('NULLs sort first in ASC',                 (SELECT name FROM s ORDER BY pts LIMIT 1) = 'c'),
 ('IS DISTINCT FROM exists (3.39+)',         (NULL IS NOT DISTINCT FROM NULL) = 1),
 -- ...
 ('UNIQUE allows several NULLs',             (SELECT count(*) FROM u) = 2);
BEGIN; CREATE TABLE gone (x); ROLLBACK;       -- DDL is transactional in SQLite too
INSERT INTO claim SELECT 'DDL rolls back', (SELECT count(*) FROM sqlite_master WHERE name = 'gone') = 0;
```

Output:

```
28 SQLite claims verified
```

20.18 Interview pattern catalogue

Most SQL interview questions are one of about twenty patterns. Recognise the pattern from the wording, write the template, then adapt it. Method and edge-case checklist: §18.1 and §19.1. Full catalogues: §18.10 and §19.11.

Pattern	Wording that gives it away	Core technique	Solved examples
Plain filter	"find rows where …", "not equal to"	`WHERE` with NULL-aware logic (`IS DISTINCT FROM`)	P1, P2, P4, 19.2.1
String and regex filter	"valid e-mail", "longer than N characters"	`~` with anchors, `char_length`	P6, P7
Row-level classification	"label each row", "is it a triangle"	`CASE`	P8, 19.10.3
Group and count	"per", "for each", "how many"	`GROUP BY` + aggregate, `HAVING`	P10, P12, P14, 19.2.2
Zero counts included	"including those with none"	`LEFT JOIN` + `count(right.col)` or `CROSS JOIN` grid	P20, 19.2.3
Distinct lists per group	"list the products sold each day"	`count(DISTINCT)`, `string_agg(DISTINCT … ORDER BY …)`	P15, P16, 19.10.2
Semi-join	"who has at least one"	`EXISTS`	P22
Anti-join	"never", "without", "no sales"	`NOT EXISTS` (not `NOT IN`)	P23, P24, P25, P26
Conditional aggregation / static pivot	"one column per status"	`count(*) FILTER (WHERE …)`, `SUM(CASE …)`	P27, P28, 19.6.1
Dynamic pivot / unpivot	"one column per store", columns unknown	`format('%I')` + `EXECUTE`, `LATERAL (VALUES …)`, `jsonb_each_text`	19.6.2, 19.6.3
Date ranges	"in February 2020", "last 30 days"	half-open `>= start AND < next_start`	P23, P29, P30, P31
As-of value	"price on that date"	latest row `<=` the date (`DISTINCT ON` or `LIMIT 1`) + default	P33
Ratio / rate	"percentage", "rate", "share"	`100.0 * …`, `NULLIF(denominator, 0)`, round last	P34, P35, P36, P37, 19.9.1, 19.9.2, 19.9.3
Nth highest	"second highest", "NULL if none"	`DISTINCT … OFFSET n-1 LIMIT 1` in a scalar subquery, or `dense_rank`	P39, 19.3.1
Top-N per group	"top 3 in each department"	`dense_rank() OVER (PARTITION BY …)` in a CTE, filter outside	19.3.2
First / last row per group	"first login device", "latest status"	`DISTINCT ON`, `row_number() = 1`	P40, 19.3.3
Ties must be kept	"all winners", "include ties"	`rank() = 1`, `WITH TIES`, `= (SELECT max …)`	P21, 19.3.8, 19.3.9
Running total / cut-off	"so far", "cumulative", "who fits"	`sum() OVER (ORDER BY …)`	P41, 19.4.1, 19.4.2
Compare with previous row	"improved", "change since last"	`lag()`	19.4.3, 19.5.6
Compare with group	"above the department average"	`avg() OVER (PARTITION BY …)`	P42, 19.7.5
Gaps and islands / streaks	"N in a row", "consecutive", "longest streak"	`value − row_number()`, or difference of two row numbers	P32, 19.5.1, 19.5.2, 19.5.3, 19.5.5

Pattern	Wording that gives it away	Core technique	Solved examples
Pairs stored once	friends, calls, matches	`UNION ALL` both directions	19.7.1, 19.7.2, 19.7.4
Relational division	"took every course"	`HAVING count(DISTINCT x) = (SELECT count(*) …)`	19.7.6
Retention / cohorts	"came back the next day"	first date per user, then join on `first + 1`	P38, 19.8.1
Histogram with empty buckets	"include zero"	`generate_series` + `LEFT JOIN`	19.8.2
Largest unique value	"largest number that appears once"	`HAVING count(*) = 1`, then `max` (NULL when none)	P9

20.19 Interview questions and answers

Concept questions (the coding problems are in Chapters 18–19). Each answer is the short version. Follow the link for the long one.

Fundamentals

1. **What is the difference between a database, a DBMS and SQL?** A database is an organised, persistent collection of related data. A DBMS is the software that stores it and controls access (queries, concurrency, recovery, security). SQL is the declarative language used to talk to a relational DBMS. → §1.2, §1.3, §1.9
2. **What are DDL, DML, DQL, DCL and TCL?** Define structure (`CREATE`, `ALTER`, `DROP`), change data (`INSERT`, `UPDATE`, `DELETE`), query (`SELECT`), grant rights (`GRANT`, `REVOKE`) and control transactions (`BEGIN`, `COMMIT`, `ROLLBACK`). → §1.9
3. **Relational vs NoSQL — how do you choose?** By access pattern and consistency need. Relational for structured data with joins, constraints and transactions. NoSQL families when a specific shape (key lookups, nested documents, huge write rates, graphs) dominates and you accept weaker guarantees. → §1.8, §16.10
4. **In what order is a `SELECT` evaluated?** FROM/JOIN → WHERE → GROUP BY → aggregates → HAVING → SELECT (windows) → DISTINCT → set operations → ORDER BY → LIMIT. That is why `WHERE` cannot see aliases and `ORDER BY` can. → §4.5
5. **What does NULL mean, and why does `x = NULL` not work?** NULL marks an unknown or missing value. Any comparison with it is UNKNOWN, and `WHERE` keeps only TRUE, so use `IS NULL` or `IS DISTINCT FROM`. → §2.5
6. **Why can `NOT IN` return no rows?** `x NOT IN (a, NULL)` means `x <> a AND x <> NULL`. The second part is UNKNOWN, so the whole condition is never TRUE. Use `NOT EXISTS`. → §6.2
7. **`WHERE` vs `HAVING`?** `WHERE` filters rows before grouping and cannot use aggregates. `HAVING` filters groups after aggregation. → §4.3
8. **`count(*)` vs `count(col)` vs `count(DISTINCT col)`?** All rows / non-NULL values / distinct non-NULL values. → §4.1
9. **`UNION` vs `UNION ALL`?** `UNION` removes duplicates (extra sort or hash). `UNION ALL` keeps everything and is cheaper. → §6.8
10. **`DELETE` vs `TRUNCATE` vs `DROP`?** Chosen rows (triggers fire, each row logged) / all rows quickly / the table itself. In PostgreSQL all three can be rolled back. → §9.6

Joins, subqueries, CTEs, windows

11. **Explain the join types.** Inner = matching pairs. Left/right = all of one side plus matches. Full = all of both. Cross = every pair. Self = a table joined to itself. → §5.2

12. **Why did my `LEFT JOIN` behave like an inner join?** A condition on the right table in `WHERE` removes the NULL-padded rows. Move it into `ON`. → §5.9
13. **What is a semi-join and an anti-join?** Semi: left rows that have a match, each returned once (`EXISTS`). Anti: left rows with no match (`NOT EXISTS`). → §5.8
14. **What is a correlated subquery?** A subquery that refers to the outer row, so conceptually it runs once per outer row. The planner can often turn it into a join. → §6.3
15. **`EXISTS` vs `IN`?** Both express a semi-join and PostgreSQL usually plans them the same way. The difference is `NOT IN` vs `NOT EXISTS` when NULLs are possible. → §6.4
16. **CTE vs subquery vs view vs materialized view?** A CTE names a step inside one statement. A subquery is inline. A view is a stored query, always current. A materialized view stores the result and must be refreshed. → §7.9
17. **How does a recursive CTE work?** The anchor produces the first rows. The recursive term runs again and again on the rows of the previous round until a round returns nothing. → §7.3
18. **Window function vs `GROUP BY`?** `GROUP BY` collapses each group into one row. A window function computes over related rows but keeps every row. → §8.1
19. **`ROW_NUMBER` vs `RANK` vs `DENSE_RANK`?** 1,2,3,4 vs 1,2,2,4 vs 1,2,2,3 on a tie for second place. → §8.5
20. **Why does my running total repeat on tied dates?** With `ORDER BY` and no frame clause the default frame is `RANGE … CURRENT ROW`, which includes all peers. Add a unique tie-breaker or use `ROWS`. → §8.3
21. **Why is `LAST_VALUE` returning the current row?** The default frame ends at the current row. Extend it to `UNBOUNDED FOLLOWING`. → §8.6
22. **How do you find the Nth highest salary?** `SELECT DISTINCT salary … ORDER BY salary DESC OFFSET n-1 LIMIT 1`, wrapped in a scalar subquery to get NULL when there is none, or `dense_rank() = n`. → P39
23. **What is the gaps-and-islands trick?** In a consecutive run, `value − row_number()` stays constant, so grouping by it groups each run. → §8.9

Design

24. **Primary key vs unique key vs foreign key?** PK: the chosen identifier, unique and not null, one per table. Unique: another identifier (alternate key), NULLs usually allowed. FK: a column that must match a key in another table. → §9.3
25. **Superkey vs candidate key?** A superkey identifies rows. A candidate key is a superkey with nothing extra: drop any of its columns and it no longer identifies rows. → §10.13
26. **Natural vs surrogate key?** Natural keys come from the business data and can change. Surrogate keys are generated and meaningless. A common default is a surrogate PK plus `UNIQUE` on the natural key. → §10.13
27. **Explain 1NF, 2NF, 3NF and BCNF.** Atomic values / no partial dependency on a composite key / no transitive dependency / every determinant is a superkey. → §11.7–§11.11
28. **What anomalies does normalization prevent?** Insertion, update and deletion anomalies, all caused by storing one fact in more than one place. → §11.6
29. **When would you denormalize?** For a measured read bottleneck, with a way to keep the copies correct (same transaction, trigger, or materialized view) and a reconciliation check. → §11.16
30. **How do you model a many-to-many relationship?** A junction table with two FKs and a composite PK (or a surrogate PK + `UNIQUE` on the pair). Relationship attributes go there too. → §10.15
31. **What is a weak entity?** An entity that cannot be identified without its owner, such as an order line within an order. Its key is the owner's key + a partial key. → §10.9

Performance

32. **How does a B-tree index speed up a query?** It keeps keys sorted in a shallow balanced tree (3–4 levels for millions of rows), so finding a key or the start of a range takes a few page reads instead of a full scan. → §12.6
33. **Clustered vs non-clustered index?** A clustered index stores the table rows in index order (InnoDB's primary key, SQL Server's clustered index). A non-clustered index is separate and points to the rows. PostgreSQL tables are heaps. `CLUSTER` reorders once but does not maintain the order. → §12.4

34. **How do you choose column order in a composite index?** Leftmost prefix rule: equality columns first, then the range or sort column. → §12.11
35. **What is a covering index?** One that contains every column the query needs, allowing an index-only scan. → §12.12
36. **Why is my index not used?** A function or cast on the column, a leading wildcard, low selectivity, stale statistics, a non-leading composite column, or a table so small that a seq scan is cheaper. → §12.13
37. **How do you read `EXPLAIN ANALYZE`?** Innermost node first. Compare estimated with actual rows, look at loops, the biggest actual times, sorts spilling to disk, and seq scans on big tables with selective filters. → §12.10, §12.17
38. **Why is `OFFSET` paging slow, and what is the fix?** The database still reads and discards all the skipped rows. Keyset pagination (`WHERE (ts, id) < (last_ts, last_id)`) jumps straight there through an index. → §12.14
39. **Nested loop vs hash join vs merge join?** Small outer input with an indexed inner / large equality joins / inputs already sorted. → §12.15
40. **What is the N+1 problem?** One query for a list, then one more query per item. Fix it with a join, `= ANY($1)` with a batch of ids, or ORM eager loading. → §12.13, §14.13

Transactions and concurrency

41. **Explain ACID.** Atomicity (all or nothing), consistency (rules hold), isolation (concurrent transactions do not interfere), durability (committed data survives crashes). → §13.1
42. **Name the isolation levels and anomalies.** Read Uncommitted, Read Committed, Repeatable Read, Serializable. Anomalies: dirty read, non-repeatable read, phantom, lost update, write skew. → §13.4
43. **What does PostgreSQL's Repeatable Read prevent that the standard does not require?** Phantoms (it is snapshot isolation), and it aborts lost updates with a serialization error. It still allows write skew. → §13.4
44. **How do you prevent lost updates?** An atomic `UPDATE … SET x = x - 1`, `SELECT … FOR UPDATE`, Repeatable Read with retry, or optimistic locking with a version column. → §13.7
45. **What is write skew?** Two transactions read the same data, then update different rows, together breaking a rule both checked. Serializable (SSI) or explicit locks prevent it. → §13.8
46. **What is MVCC?** Writers create new row versions, and each transaction reads the versions visible in its snapshot, so readers do not block writers. Old versions are removed by VACUUM. → §13.14
47. **Optimistic vs pessimistic locking?** Pessimistic locks before touching data (waits, deadlocks possible). Optimistic works freely and checks for conflicts at commit (aborts and retries). Low contention favours optimistic. → §13.9
48. **How are deadlocks handled?** The database detects the wait cycle and aborts one transaction (40P01). Prevent them with a consistent lock order and short transactions, and retry in code. → §13.11
49. **How do you build a job queue in SQL?** `SELECT … FOR UPDATE SKIP LOCKED LIMIT 1` inside a transaction, so each worker takes a different unlocked row. → §13.12

Security, scaling, beyond

50. **How do you prevent SQL injection?** Parameterized queries (bind variables). Never concatenate input. Allow-list identifiers. Least-privilege roles. → §14.9
51. **Partitioning vs sharding vs replication?** Partitioning splits a table inside one server. Sharding puts partitions on different servers. Replication keeps copies of the same data for availability and read scaling. → §15.2, §15.3, §15.5
52. **Explain the CAP theorem.** During a network partition a distributed store must choose between consistency (refuse some requests) and availability (answer, possibly stale). PACELC adds that without a partition you trade latency against consistency. → §15.7
53. **Synchronous vs asynchronous replication?** Sync waits for the replica before commit (no loss, higher latency). Async commits first (fast, may lose recent writes on failover). → §15.5
54. **What is the difference between a backup and a replica?** A replica copies every change, including mistakes, within seconds. A backup is a point-in-time copy you can restore from. PITR combines a

base backup with archived WAL. → §15.11, §9.12

20.20 Glossary (A–Z)

Short definitions of the terms used across the book, in plain words. The last column links to the section that explains each term with examples. The terms from the course's key-term review cards (the problems that made databases necessary, ER notation, keys, functional dependencies, normal forms, modeling levels, and the case-study judgement calls) are included, with the judgement calls summarised as a table in 20.10.

298 terms. Jump to: 0–9 · A · B · C · D · E · F · G · H · I · J · K · L · M · N · O · P · Q · R · S · T · U · V · W

0–9

Term	Meaning	See
1NF (first normal form)	Every cell holds a single atomic value and there are no repeating groups of columns.	§11.7
2NF (second normal form)	1NF, and no non-prime attribute depends on only part of a composite candidate key.	§11.8
2PC (two-phase commit)	Distributed commit protocol: the coordinator asks every participant to prepare, then tells all to commit or all to abort. Blocks if the coordinator fails after prepare.	§15.9
2PL (two-phase locking)	A transaction acquires locks in a growing phase and releases them in a shrinking phase, never acquiring after releasing. Guarantees serializable schedules.	§13.10
3NF (third normal form)	2NF, and for every FD X → A either X is a superkey or A is a prime attribute. Removes transitive dependencies.	§11.9
4NF (fourth normal form)	BCNF, and every non-trivial multivalued dependency has a superkey on its left side.	§11.12
5NF (fifth normal form)	Every join dependency is implied by the candidate keys. Also called project-join normal form.	§11.13

A

Term	Meaning	See
ACID	Atomicity, consistency, isolation, durability: the four guarantees of a database transaction.	§13.1
Adjacency list	Storing a tree or graph as rows that each point to their parent (a `parent_id` column).	§7.4
Aggregate function	A function that turns many input rows into one value, such as `count`, `sum`, `avg`, `min`, `max`.	§4.1
Alias	A temporary name for a column (`AS total`) or a table (`FROM orders o`) within one query.	§2.1
Alternate key	Any candidate key other than the one picked as primary key. Enforced with `UNIQUE`.	§10.13
ALTER TABLE	DDL statement that changes an existing table: add, drop, rename or retype columns, add constraints.	§9.5
Analytic function	Another name for a window function (common in Oracle documentation).	§8.1
ANALYZE	Command that samples a table and refreshes the planner's statistics.	§12.16
Anchor (recursive CTE)	The non-recursive first part of a recursive CTE, which produces the starting rows.	§7.3
Anomaly (data)	An insertion, update or deletion problem caused by storing the same fact in more than one row.	§11.6
Anomaly (isolation)	A result that could not happen if transactions ran one at a time, such as a dirty read or write skew.	§13.4
Anti-join	Returns rows of one table that have no match in another. Written `NOT EXISTS` or `LEFT JOIN … IS NULL`.	§5.8
ANY / SOME / ALL	Compare a value with a set: true if the comparison holds for at least one element (`ANY`, `SOME`) or for every element (`ALL`).	§6.5
Armstrong's axioms	Reflexivity, augmentation and transitivity: sound and complete rules for deriving functional dependencies.	§11.3
Array (PostgreSQL)	A column type holding an ordered list of values of one type, indexed from 1.	§14.5
Atomicity	A transaction's changes happen completely or not at all.	§13.2
Attribute	A named property of an entity or a column of a relation.	§10.6
Attribute closure (X^+)	Every attribute functionally determined by the set X under a given set of FDs.	§11.4
Autocommit	Mode in which every statement is its own transaction and commits immediately.	§13.1

B

Term	Meaning	See
BASE	Basically available, soft state, eventually consistent: the looser guarantee model of many NoSQL systems.	§16.1
BCNF (Boyce–Codd normal form)	For every non-trivial FD X → Y, X is a superkey.	§11.11
BEGIN / COMMIT / ROLLBACK	Start a transaction, make its changes permanent, or discard them.	§13.1
BETWEEN	Inclusive range test: `x BETWEEN a AND b` means `x >= a AND x <= b`.	§2.4
Bitmap scan	Plan that collects matching row locations from one or more indexes into a bitmap, then reads the table pages in physical order.	§12.10
Bloom filter	Compact probabilistic set that answers "definitely not present" or "probably present".	§12.8
BRIN index	Block range index: stores min/max per range of table pages. Tiny and effective when values follow physical order, such as time.	§12.9
B-tree / B+ tree	Balanced, shallow, sorted tree index. In a B+ tree all keys live in linked leaf pages, which makes range scans cheap. PostgreSQL's default index.	§12.6
Buffer pool (shared buffers)	Memory area where the DBMS caches data pages read from disk.	§1.4

C

Term	Meaning	See
Candidate key	A minimal set of attributes that uniquely identifies every row.	§10.13
CAP theorem	If the network splits, a replicated store can stay fully consistent or stay fully available, but not both.	§15.7
Cardinality (ER)	The maximum number of instances on each side of a relationship: 1:1, 1:N or M:N.	§10.7
Cardinality estimate	The planner's guess of how many rows a plan node will produce.	§12.16
Cardinality (relation)	The number of rows in a relation.	§10.12
Cartesian product	Every row of one table paired with every row of another.	§5.1
CASCADE	Referential action: deleting or updating a parent row does the same to its child rows.	§9.3
CASE	Conditional expression returning the first `THEN` value whose `WHEN` is true, else the `ELSE` value (or NULL).	§3.5
Catalog (data dictionary)	System tables describing all database objects: tables, columns, types, indexes, privileges.	§1.2
CDC (change data capture)	Streaming committed row changes out of a database, usually by reading its log, into other systems.	§16.10
CHECK constraint	A row-level rule that rejects any row for which the expression is FALSE.	§9.3
Checkpoint	Point at which all dirty pages are flushed to disk, bounding how much log must be replayed after a crash.	§15.9
Chen notation	ER notation that draws entities as rectangles, attributes as ellipses and relationships as diamonds.	§10.5
Clustered index	An index whose order is the physical order of the table rows (InnoDB primary key, SQL Server clustered index).	§12.4
COALESCE	Returns the first non-NULL argument.	§3.5
Collation	Rules that decide how text is compared and sorted, including case sensitivity.	§2.7
Column store (columnar)	Storage that keeps each column's values together, making scans and compression of a few columns efficient.	§12.2
Composite index	An index on several columns. Usable for filters on a leftmost prefix of its columns.	§12.11
Composite key	A key made of two or more columns.	§10.13
Conceptual model	Business-level model of entities and relationships, without types or DBMS details.	§10.2
Concurrency control	The mechanisms (locking, timestamps, MVCC, validation) that keep concurrent transactions correct.	§13.9
Connection pool	A set of reusable open database connections shared by application threads.	§14.12
Consensus	Agreement of a group of nodes on one value or log order despite failures, as in Raft and Paxos.	§15.8
Consistency (ACID)	Every committed transaction leaves all constraints and business rules satisfied.	§13.3
Consistency model	The guarantee a distributed store gives about what reads can see: strong, causal, eventual and others.	§15.6

Term	Meaning	See
Consistent hashing	Placing nodes and keys on a hash ring so that adding a node moves only a small share of keys.	§15.3
Constraint	A rule enforced by the DBMS on the data: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, EXCLUDE.	§9.3
Correlated subquery	A subquery that refers to a column of the outer query and is logically evaluated per outer row.	§6.3
Cost (planner)	The planner's estimate of the work of a plan, in arbitrary units based on page reads and CPU.	§12.10
Covering index	An index containing every column a query needs, allowing an index-only scan.	§12.12
CROSS JOIN	Join that returns the Cartesian product.	§5.1
Crow's foot notation	ER notation that draws cardinality and optionality with symbols at the ends of relationship lines.	§10.5
CTE (common table expression)	A named subquery defined with `WITH` and used by the statement that follows.	§7.1
CUBE	Grouping extension that produces subtotals for every combination of the listed columns.	§4.6
Cursor	A pointer for fetching a query result a batch of rows at a time.	§14.10

D

Term	Meaning	See
Database	An organised, persistent, self-describing collection of related data.	§1.2
Data independence	Changing how data is stored (physical) or parts of the schema (logical) without changing applications.	§1.3
Data model	A description of the data's entities, attributes, relationships and rules.	§10.1
Data-modifying CTE	A `WITH` step containing `INSERT`, `UPDATE` or `DELETE … RETURNING` (PostgreSQL).	§7.2
DBMS	Software that defines, stores, queries, protects and shares a database.	§1.3
DCL	Data control language: `GRANT` and `REVOKE`.	§1.9
DDL	Data definition language: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`.	§1.9
Deadlock	Two or more transactions each waiting for a lock held by another, so none can proceed.	§13.11
Decomposition	Splitting a relation into smaller ones during normalization.	§11.10
Degree (ER)	The number of entity types taking part in a relationship: unary, binary, ternary.	§10.7
Degree (relation)	The number of attributes (columns) of a relation.	§10.12
DELETE	DML statement that removes the rows matching its `WHERE` clause.	§9.10
Denormalization	Deliberately storing redundant data to speed up reads, with a mechanism to keep it correct.	§11.16
DENSE_RANK	Ranking function that gives ties the same rank and leaves no gaps (1, 2, 2, 3).	§8.5
Dependency preservation	Every functional dependency can be checked within a single table after decomposition.	§11.10
Derived attribute	An attribute that can be computed from others, such as age from birth date.	§10.6
Derived table	A subquery in the `FROM` clause.	§6.6
Determinant	The left side of a functional dependency.	§11.1
Dirty read	Reading another transaction's uncommitted change.	§13.4
DISTINCT	Removes duplicate rows from a result. `DISTINCT ON (cols)` keeps the first row per group (PostgreSQL).	§2.6
DML	Data manipulation language: `INSERT`, `UPDATE`, `DELETE`, `MERGE`.	§1.9
Document database	NoSQL store that keeps each record as a nested document, usually JSON.	§16.3
Domain	The set of allowed values for an attribute.	§10.12
DQL	Data query language: `SELECT`.	§1.9
DROP	DDL statement that removes a database object entirely.	§9.6
Durability	Once committed, a transaction's changes survive crashes.	§13.5

E

Term	Meaning	See
Entity	A distinguishable thing the database stores facts about, such as a customer or an order.	§10.6
Entity integrity	No primary-key value may be NULL.	§10.14
Equi-join	A join whose condition is equality between columns.	§5.3
ER diagram	A diagram of entities, attributes and relationships.	§10.5
Eventual consistency	If writes stop, all replicas eventually return the same value.	§15.6
EXCEPT	Set operation returning rows of the first query that do not appear in the second.	§6.10
EXCLUDE constraint	PostgreSQL constraint that forbids two rows whose values conflict under given operators, such as overlapping ranges.	§17.2
EXISTS	True if the subquery returns at least one row.	§6.4
EXPLAIN / EXPLAIN ANALYZE	Show the chosen query plan, and with `ANALYZE` run it and report actual rows and times.	§12.10
Expression index	An index on the result of an expression, such as `lower(email)`.	§12.12

F

Term	Meaning	See
Failover	Promoting a replica to primary when the primary fails.	§15.10
Fan-out	Row multiplication when a row joins to several matching rows, often inflating sums.	§5.9
Fencing	Preventing a deposed leader from continuing to write after failover.	§15.8
FETCH FIRST	Standard syntax for limiting rows: `FETCH FIRST n ROWS ONLY` or `WITH TIES`.	§2.8
File organization	How records are arranged in a file: heap, sorted, hashed or clustered.	§12.3
FILTER clause	Restricts the rows fed to one aggregate: `count(*) FILTER (WHERE …)`.	§4.4
Foreign key	Columns whose values must match a key in a referenced table (or be NULL).	§9.3
Frame (window frame)	The subset of the partition that an aggregate or value window function sees for the current row.	§8.4
FULL OUTER JOIN	Returns all rows from both sides, padding with NULLs where there is no match.	§5.6
Full-text search	Searching documents by words with stemming and ranking. `tsvector @@ tsquery` in PostgreSQL.	§14.6
Functional dependency	`X → Y`: any two rows with equal X values have equal Y values.	§11.1

G

Term	Meaning	See
Gaps and islands	Problems about runs of consecutive values or dates, solved by grouping on `value − row_number()`.	§8.9
generate_series	PostgreSQL function that returns a series of numbers or timestamps as rows.	§7.6
GIN index	Generalized inverted index: maps each element (word, key, array item) to the rows that contain it.	§12.9
GiST index	Generalized search tree for overlap and nearest-neighbour queries on ranges, geometry and more.	§12.9
Graph database	NoSQL store of nodes and edges optimised for traversing relationships.	§16.5
GROUP BY	Groups rows that share values so aggregates are computed per group.	§4.2
GROUPING SETS	Computes several groupings in one query, such as per region, per product and a grand total.	§4.6
GROUPS frame	A window frame measured in peer groups rather than rows or values.	§8.4

H

Term	Meaning	See
Hash index	Index that hashes keys into buckets. Supports equality lookups only.	§12.7
Hash join	Join that builds a hash table on one input and probes it with the other. Equality only.	§12.15
Hash partitioning	Assigning rows to partitions by a hash of the key, spreading them evenly.	§15.2
HAVING	Filters groups after aggregation.	§4.3
Heap	Unordered table storage: new rows go wherever there is space.	§12.1
High availability	Keeping a service running through failures, usually with replicas and automatic failover.	§15.10
Horizontal scaling	Adding more machines (scale out) rather than a bigger one.	§15.1

I

Term	Meaning	See
Identity column	A column whose values are generated automatically: `GENERATED ALWAYS AS IDENTITY`.	§9.4
IN	True if a value equals any element of a list or subquery result.	§6.2
Index	An auxiliary structure that finds rows by key without scanning the whole table.	§12.4
Index-only scan	Answering a query from the index alone, without visiting table pages.	§12.10
Index scan	Walking an index to find matching entries, then fetching each row from the table.	§12.10
In-memory database	A database that keeps its working data in RAM and persists it with snapshots and logs.	§16.6
INNER JOIN	Returns only pairs of rows that satisfy the join condition.	§5.3
INSERT	DML statement that adds rows.	§9.8
INTERSECT	Set operation returning rows present in both queries.	§6.9
Isolation	Concurrent transactions do not observe each other's intermediate states.	§13.4
Isolation level	How strongly a transaction is isolated: Read Uncommitted, Read Committed, Repeatable Read or Serializable.	§13.4

J

Term	Meaning	See
Join	Combining rows of two tables according to a condition.	§5.2
Join dependency	A relation equals the join of some of its projections.	§11.13
JSONB	PostgreSQL's binary JSON type, which supports indexing and containment queries.	§14.4
Junction table	A table that implements a many-to-many relationship with foreign keys to both sides.	§10.15

K

Term	Meaning	See
Keyset pagination	Paging by remembering the last key seen (`WHERE key > :last`) instead of using `OFFSET`.	§12.14
Key-value store	NoSQL store that maps keys to values with get and put operations.	§16.2

L

Term	Meaning	See
LAG / LEAD	Window functions returning a value from a row before or after the current one.	§8.6
LATERAL	A subquery in `FROM` that may refer to columns of tables listed before it.	§6.7
Leader election	Choosing one node to coordinate writes, typically via a consensus system.	§15.8
Leftmost prefix rule	A composite index on (a, b, c) helps filters on a, (a, b) or (a, b, c), not on b alone.	§12.11
LEFT OUTER JOIN	Returns every left row, with matching right rows or NULLs.	§5.4
LIKE / ILIKE	Pattern matching with `%` and `_`. `ILIKE` is case-insensitive (PostgreSQL).	§2.4
LIMIT / OFFSET	Return at most n rows, after skipping k rows.	§2.8
List partitioning	Assigning rows to partitions by a list of discrete values, such as region codes.	§15.2
Lock	A marker that restricts other transactions' access to a row or table until released.	§13.10
Logical model	Model with attributes, keys and normalized relations, independent of a particular DBMS.	§10.2
Logical query processing order	The conceptual order in which the clauses of a SELECT are evaluated, starting with FROM.	§4.5
Lossless join	A decomposition whose pieces join back to exactly the original rows.	§11.10
Lost update	Two read-modify-write transactions where the second silently overwrites the first.	§13.7
LSM tree	Log-structured merge tree: writes go to memory and sorted files that are merged in the background.	§16.4

M

Term	Meaning	See
Materialized view	A stored query result that must be refreshed to reflect new data.	§7.8
MERGE	Standard statement that inserts, updates or deletes target rows based on a source (PostgreSQL 15+).	§9.11
Merge join	Join that walks two inputs sorted on the join key.	§12.15
Metadata	Data about data, such as table and column definitions.	§1.2
Minimal cover	An equivalent set of FDs with single-attribute right sides, no extraneous attributes and no redundant FDs.	§11.5
Multivalued attribute	An attribute that can hold several values for one entity, such as phone numbers.	§10.6
Multivalued dependency	`X ↠ Y`: for each X, the set of Y values is independent of the other attributes.	§11.12
MVCC	Multiversion concurrency control: keep several row versions so readers see a snapshot and do not block writers.	§13.14

N

Term	Meaning	See
N+1 query problem	Running one query for a list, then one more per item, instead of a single set-based query.	§12.13
Natural join	A join on all columns with the same name. Risky because it changes silently when columns are added.	§5.10
Natural key	A key taken from real-world data, such as an ISBN or email.	§10.13
Nested loop join	For each row of the outer input, look up matches in the inner input.	§12.15
NewSQL	Distributed databases that keep SQL and ACID while scaling out, such as Spanner and CockroachDB.	§16.7
Non-repeatable read	Reading the same row twice in a transaction and getting different values.	§13.4
Normalization	Organising tables so each fact is stored once, guided by functional dependencies and normal forms.	§11.6
NoSQL	Non-relational databases: key-value, document, wide-column and graph stores.	§16.1
NOT NULL	Constraint forbidding NULL in a column.	§9.3
NTILE	Window function that splits ordered rows into n roughly equal buckets.	§8.5
NULL	A marker for a missing or unknown value. Comparisons with it yield UNKNOWN.	§2.5
NULLIF	Returns NULL if its two arguments are equal, otherwise the first.	§3.5

O

Term	Meaning	See
OLAP	Online analytical processing: large scans and aggregates over historical data.	§11.16
OLTP	Online transaction processing: many small reads and writes of current data.	§11.16
ON CONFLICT	PostgreSQL clause that turns an `INSERT` into an upsert or skips conflicting rows.	§9.11
ON DELETE action	What happens to child rows when a parent is deleted: CASCADE, SET NULL, SET DEFAULT, RESTRICT, NO ACTION.	§9.3
Optimistic concurrency control	Proceed without locks, validate at commit, and abort or retry on conflict.	§13.9
Optimizer (planner)	The DBMS component that chooses an efficient execution plan for a query.	§1.4
ORDER BY	Sorts the result. The only way to guarantee row order.	§2.7
ORM	Object-relational mapper: a library that maps classes to tables and generates SQL.	§14.13
Orphan row	A child row whose foreign key points to a parent that does not exist.	§10.14
OUTER JOIN	LEFT, RIGHT or FULL join, which keeps unmatched rows padded with NULLs.	§5.2

P

Term	Meaning	See
PACELC	Extension of CAP: under Partition choose Availability or Consistency, Else choose Latency or Consistency.	§15.7
Page	Fixed-size unit of storage and I/O (8 KB in PostgreSQL).	§12.1
Parameterized query	A query whose values are sent separately from the SQL text as bind parameters, preventing injection.	§14.9
Partial dependency	A non-prime attribute determined by a proper subset of a composite key.	§11.8
Partial index	An index that covers only rows matching a `WHERE` condition.	§12.12
Participation	Whether every entity instance must take part in a relationship (total) or may not (partial).	§10.7
PARTITION BY (window)	Splits rows into independent groups for a window function without collapsing them.	§8.2
Partitioning	Splitting one table into smaller physical pieces by range, list or hash.	§15.2
Partition pruning	Skipping partitions that cannot contain matching rows.	§15.2
Peer rows	Rows with equal values of the window `ORDER BY` key.	§8.3
Pessimistic concurrency control	Lock data before using it so conflicting transactions wait.	§13.9
Phantom read	Re-running a query in a transaction returns a different set of rows because of inserts or deletes.	§13.4
Physical model	Model with exact types, indexes, partitions and storage for a specific DBMS.	§10.2
PITR (point-in-time recovery)	Restoring a base backup and replaying archived WAL up to a chosen moment.	§9.12
Pivot	Turning row values into columns, usually with conditional aggregation.	§14.7
PL/pgSQL	PostgreSQL's procedural language for functions, procedures and triggers.	§14.1
Polyglot persistence	Using several kinds of database in one system, each for what it does best.	§16.10
Predicate lock	A lock on a condition rather than on existing rows, used to prevent phantoms.	§13.10
Prepared statement	A statement parsed once and executed many times with different parameters.	§14.9
Primary key	The chosen candidate key: unique and not NULL, one per table.	§9.3
Prime attribute	An attribute appearing in some candidate key.	§10.13
Privilege	A permission such as SELECT or INSERT on an object, granted to a role.	§14.8
Procedure (stored)	Named server-side routine called with `CALL`. Can control transactions.	§14.2
psql	PostgreSQL's command-line client.	§1.10

Q

Term	Meaning	See
Query plan	The tree of operations (scans, joins, sorts) the executor runs for a query.	§12.10
Quorum	The number of replicas that must acknowledge an operation. `R + W > N` makes reads see the latest write.	§15.5

R

Term	Meaning	See
Raft	A consensus algorithm with an elected leader that replicates a log to a majority.	§15.8
RANGE frame	A window frame defined by value distance from the current row's `ORDER BY` value. Includes peers.	§8.4
Range partitioning	Assigning rows to partitions by ranges of a key, usually dates.	§15.2
RANK	Ranking function that gives ties the same rank and skips the next ranks (1, 2, 2, 4).	§8.5
Read Committed	Isolation level where each statement sees only data committed before it began. PostgreSQL's default.	§13.4
Read replica	A copy of the database that serves read queries.	§15.5
Read skew	Seeing different parts of the data at different moments within one transaction.	§13.4
Recovery	Restoring a consistent state after a crash, using the log to redo committed and undo uncommitted work.	§15.9
Recursive CTE	A CTE that refers to itself, used for hierarchies, graphs and series.	§7.3
Redundancy	The same fact stored more than once.	§11.6
Referential integrity	Every foreign-key value matches an existing referenced key (or is NULL).	§10.14
Relation	The formal name for a table: a set of tuples over a set of attributes.	§10.12
Relational division	Finding rows related to every member of another set, such as students who took all courses.	§19.7
Relational model	Data as relations of atomic values, linked by values and queried declaratively (Codd, 1970).	§1.6
Repeatable Read	Isolation level where rows read once keep their values. In PostgreSQL, snapshot isolation.	§13.4
Replication	Keeping copies of the same data on several servers.	§15.5
RETURNING	Clause that returns the rows affected by `INSERT`, `UPDATE` or `DELETE`.	§9.8
RIGHT OUTER JOIN	Returns every right row, with matching left rows or NULLs.	§5.5
Role	A PostgreSQL user or group that owns objects and holds privileges.	§14.8
ROLLUP	Grouping extension that adds hierarchical subtotals and a grand total.	§4.6
Row-level security	Policies that filter which rows a role may read or write.	§14.8
ROW_NUMBER	Window function giving each row a unique sequential number in its partition.	§8.5
ROWS frame	A window frame counted in physical rows.	§8.4
RPO / RTO	Recovery point objective (data you may lose) and recovery time objective (downtime you may have).	§9.12

S

Term	Meaning	See
Sargable	A predicate the database can answer with an index seek, such as `col >= value` with no function on `col`.	§12.13
Savepoint	A marker inside a transaction you can roll back to without abandoning the whole transaction.	§13.2
Scalar subquery	A subquery that returns one value. Zero rows give NULL, more than one row is an error.	§6.1
Schema	The structure of a database (tables, columns, constraints), or a PostgreSQL namespace that groups objects.	§9.2
Schema-on-read / on-write	Structure checked when data is read (typical of NoSQL) or when it is written (relational).	§1.8
Selectivity	The fraction of rows a predicate keeps. Low fractions favour index scans.	§12.16
SELF JOIN	Joining a table to itself under two aliases.	§5.7
Semi-join	Returns rows of one table that have at least one match in another, each once.	§5.8
Sequence	A database object that hands out increasing numbers.	§9.4
Sequential scan	Reading every page of a table.	§12.10
Serializable	The strongest isolation level: the outcome equals some serial order of the transactions.	§13.8
Serialization failure	Error 40001: the database aborted a transaction to preserve isolation. Retry it.	§13.7
Set operation	UNION, INTERSECT or EXCEPT, which combine the results of two queries.	§6.8
Sharding	Placing partitions of the data on different servers.	§15.3
Shard key	The column that decides which shard stores a row.	§15.3
Snapshot isolation	Each transaction reads from a consistent snapshot taken at its start. Allows write skew.	§13.4
Soft delete	Marking a row as deleted (`deleted_at`) instead of removing it.	§17.14
SQL	Structured Query Language: the declarative standard language of relational databases.	§1.9
SQL injection	An attack that smuggles SQL into a query through unescaped input.	§14.9
SSI	Serializable snapshot isolation: snapshot isolation plus tracking of read-write conflicts, aborting when a cycle is possible.	§13.8
Statistics	Per-column summaries (distinct counts, common values, histograms) used by the planner.	§12.16
Storage engine	The DBMS layer that stores pages and rows on disk.	§1.4
Subquery	A query nested inside another statement.	§6.1
Superkey	Any set of attributes that uniquely identifies rows.	§10.13
Surrogate key	An artificial key with no business meaning, such as an identity number or UUID.	§10.13
Synchronous replication	The primary waits for a replica to confirm before reporting commit.	§15.5

T

Term	Meaning	See
Table	The SQL form of a relation: rows and named, typed columns.	§1.6
TCL	Transaction control language: `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`.	§1.9
Three-valued logic	SQL logic with TRUE, FALSE and UNKNOWN.	§2.5
Time-series database	A store optimised for timestamped measurements, retention and downsampling.	§16.8
Timestamp ordering	Concurrency control that orders conflicting operations by transaction timestamps.	§13.13
TOAST	PostgreSQL's mechanism for compressing or moving large values out of the main table page.	§12.1
Transaction	A unit of work of one or more statements that succeeds or fails as a whole.	§13.1
Transitive dependency	A → B and B → C where B is not a key, so C depends on the key only through B.	§11.9
Trigger	A function run automatically before or after changes to a table.	§14.3
Trivial FD	An FD whose right side is contained in its left side.	§11.2
TRUNCATE	Quickly removes all rows from a table.	§9.6
Tuple	The formal name for a row.	§10.12

U

Term	Meaning	See
UNION / UNION ALL	Combine two results, removing duplicates (`UNION`) or keeping them (`UNION ALL`).	§6.8
UNIQUE constraint	Forbids duplicate non-NULL values in a column or column set.	§9.3
Unpivot	Turning columns into rows.	§14.7
UPDATE	DML statement that changes column values in matching rows.	§9.9
Update anomaly	A change to a fact must be repeated in many rows, and missing one leaves the data inconsistent.	§11.6
Upsert	Insert a row, or update it if it already exists.	§9.11
UUID	128-bit identifier that can be generated anywhere without coordination.	§9.4

V

Term	Meaning	See
VACUUM	PostgreSQL maintenance that reclaims dead row versions and updates the visibility map.	§13.14
Vertical scaling	Moving to a bigger machine (scale up).	§15.1
View	A named, stored query that behaves like a table and holds no data of its own.	§7.7

W

Term	Meaning	See
WAL (write-ahead log)	A log of changes written and flushed before the data pages, used for durability, recovery and replication.	§13.5
Weak entity	An entity identified only together with its owner entity.	§10.9
WHERE	Filters rows before grouping. Keeps only rows where the condition is TRUE.	§2.2
Wide-column store	NoSQL store with partition keys and sorted clustering columns, such as Cassandra.	§16.4
Window function	A function computed over a set of rows related to the current row, without collapsing them.	§8.1
WITH TIES	Extends `FETCH FIRST n` to include rows tied with the last one.	§2.8
Write skew	Two transactions read overlapping data and write different rows, together breaking a rule each checked.	§13.8

Chapter 20 TL;DR

- **Evaluation order** explains most errors: FROM → WHERE → GROUP BY → HAVING → SELECT (windows) → DISTINCT → set operations → ORDER BY → LIMIT (20.1).
- **NULL** is unknown: `WHERE` keeps only TRUE, `NOT IN` with a NULL returns nothing, `count(col)` and other aggregates skip NULLs, and `sum` over no rows is NULL (20.2, 20.4).
- **PostgreSQL arithmetic**: `7 / 2 = 3`, so write `100.0 *` before dividing and guard with `NULLIF(x, 0)` (20.3).
- **Joins**: right-table filters of a `LEFT JOIN` go in `ON`. `NOT EXISTS` is the safe anti-join. Watch fan-out (20.5, 20.6).
- **Windows**: `row_number` / `rank` / `dense_rank` differ only on ties. The default frame with `ORDER BY` is `RANGE … CURRENT ROW`, so give `last_value` a full frame (20.8).
- **Design**: model entities, keys and relationships first, normalize to 3NF/BCNF, denormalize only with a reason and a way to keep copies right (20.10).
- **Indexes**: B-tree by default, equality columns before range columns, no functions on indexed columns, keyset instead of deep `OFFSET`, and compare estimated with actual rows in `EXPLAIN ANALYZE` (20.11).
- **Transactions**: PostgreSQL defaults to Read Committed. Repeatable Read is snapshot isolation (write skew still possible). Serializable (SSI) prevents everything but needs retries on 40001 (20.12).
- **Dialects** differ most on paging, concatenation, NULL sorting, upsert, `RETURNING`, integer division and transactional DDL (20.16).
- About 100 PostgreSQL claims and 28 SQLite claims on these sheets are checked by runnable scripts (20.17). The pattern catalogue (20.18) and the fifty Q&A (20.19) link back to the full explanations.

This is the end of the SQL Handbook. For more on the design side of data systems, see the DDIA notes, and for how databases fit into whole systems, the case studies in Chapter 17.